

4.CPU原理与设计

- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期流水线处理器设计
- 4.7 流水线冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例

4.5 流水线技术

■ **流水线：**使多条指令重叠执行的技术。

■ **洗衣房的流水线**

▪ 处理器：洗衣房

▪ 4条指令：洗衣服 

▪ 指令过程：洗涤→烘干→熨整

▪ 指令各阶段延迟

▪ 洗衣：30分钟



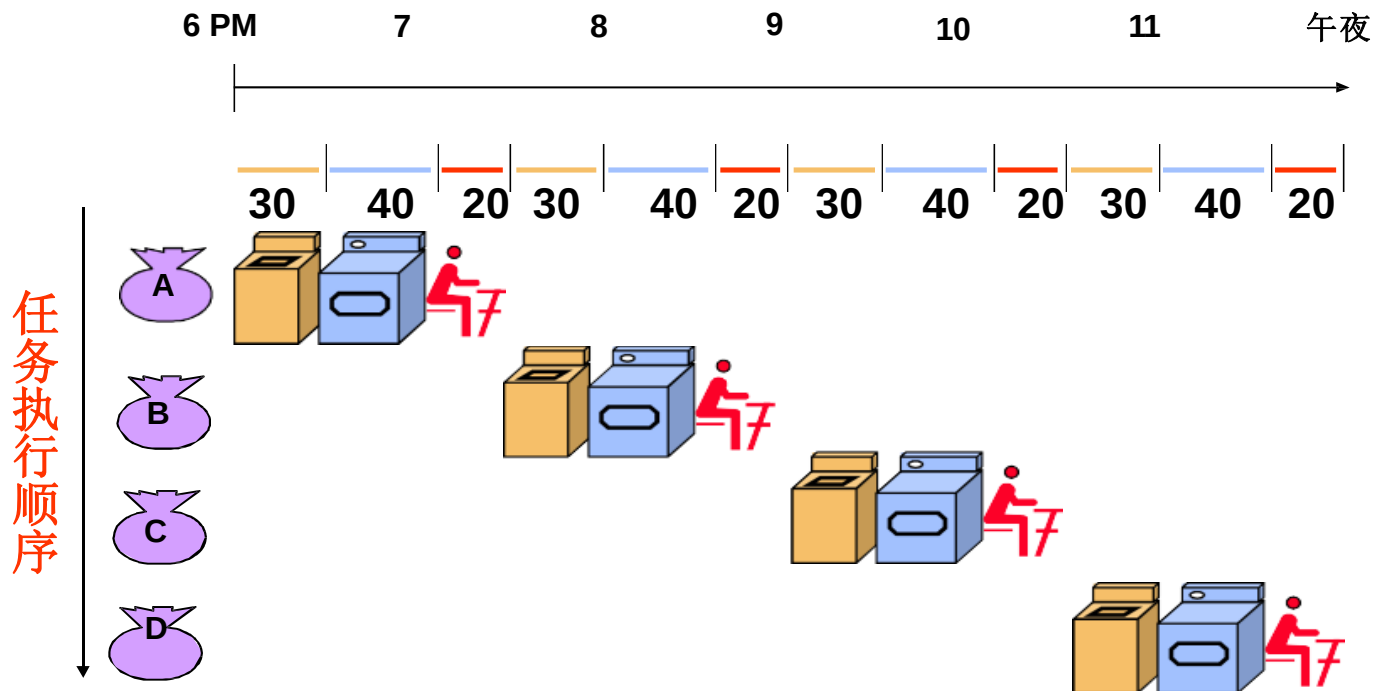
▪ 烘干：40分钟



▪ 熨整：20分钟

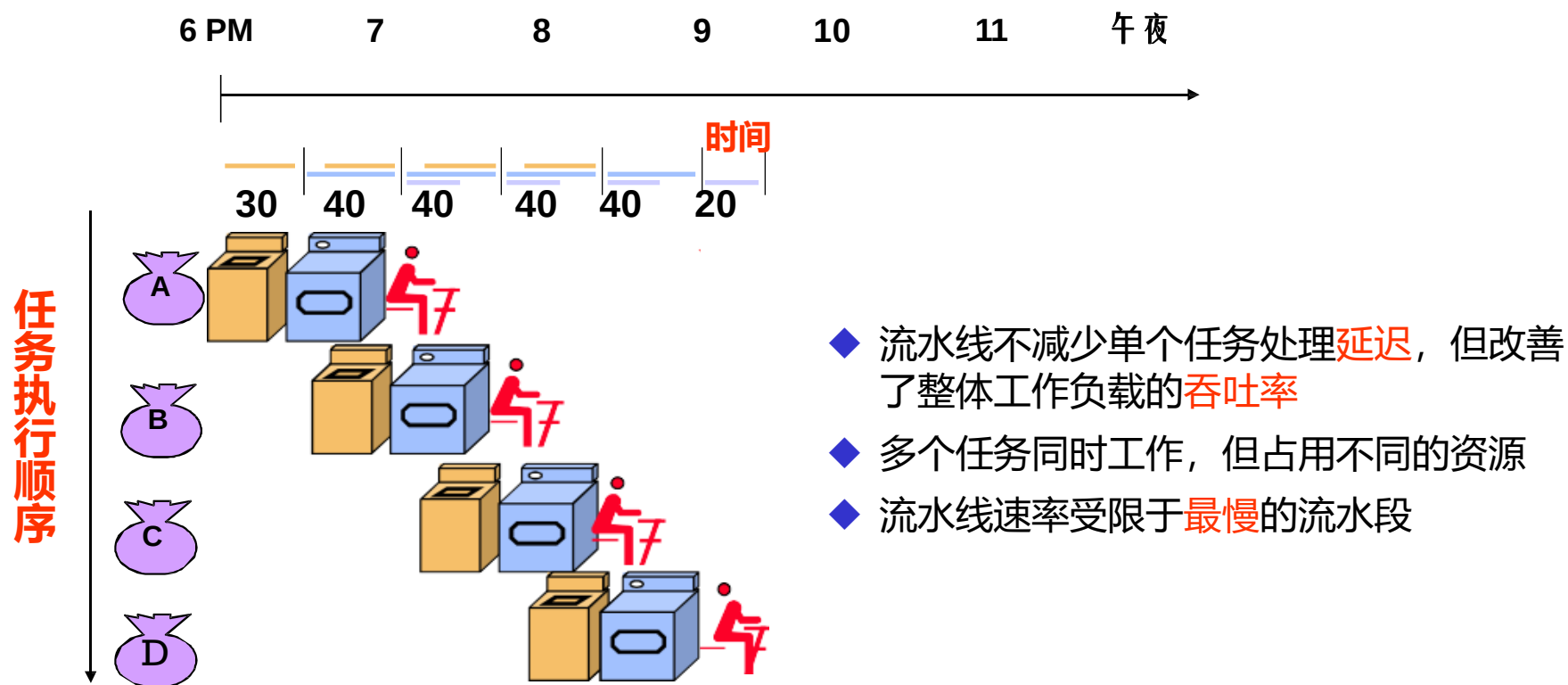


串行洗衣房



- 串行洗衣房：6个小时完成4个任务

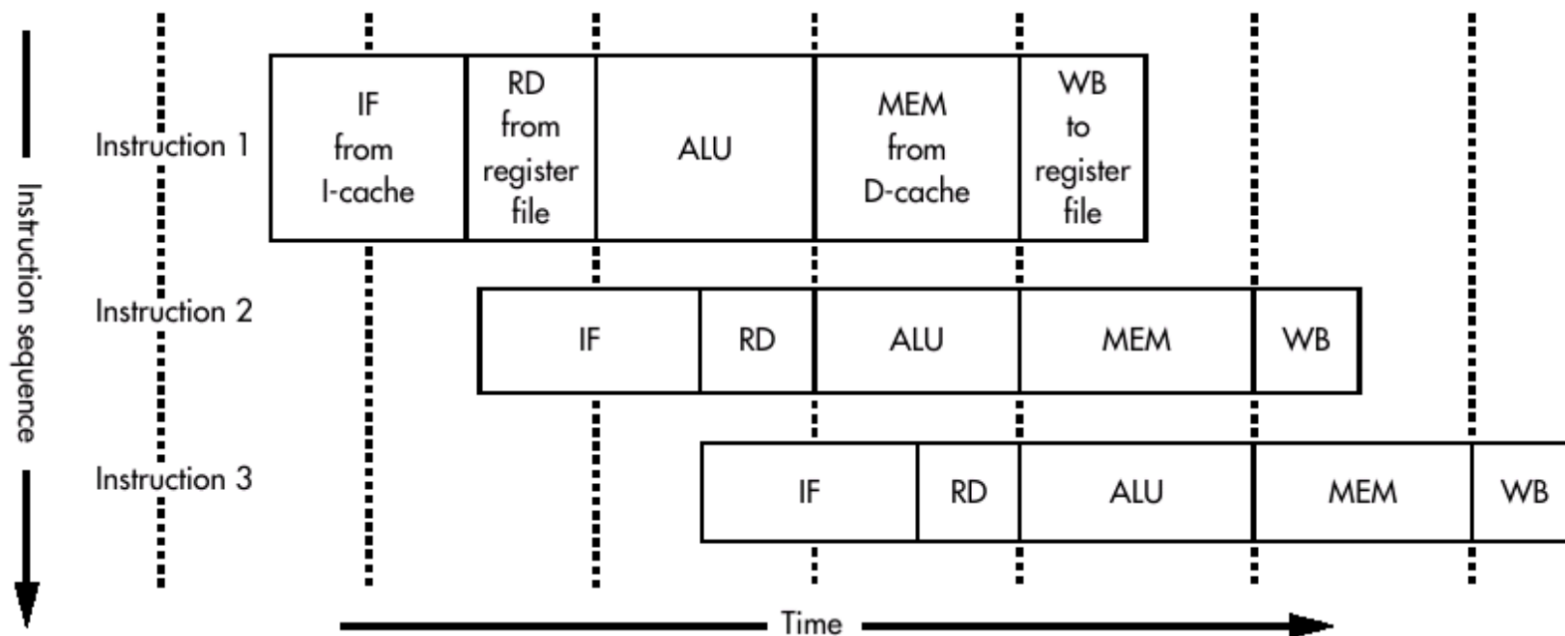
流水化洗衣房



流水化洗衣房: 3.5小时完成4个任务

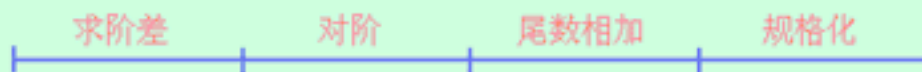
流水线技术

- MIPS的五级流水
 - 取指、读寄存器、计算、访存、写回寄存器



流水线举例

浮点加法流水线



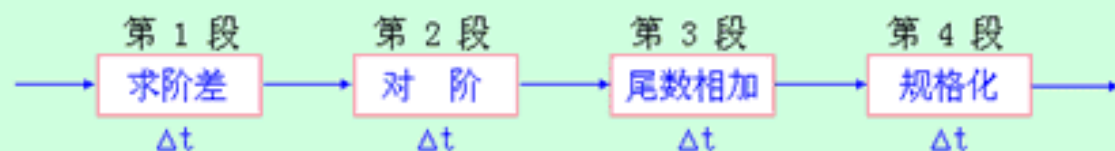
浮点加法指令的解释过程



虽然每个加法操作所需的总时间都是 $4\Delta t$ ，但在加法器的输出端，却是每隔一个 Δt 就给出一个加法结果。

流水线的时-空图

流水线的工作过程常用时(间)-空(间)图来描述。下面以浮点加法流水线的时-空图为例来说明。



浮点加法流水线



流水线分类

■ 按处理级别分类

- **功能部件级流水线**：也称运算操作流水线，在实现较为复杂的运算时采用
- **指令级流水线**：将一条指令执行过程分为多个阶段，每个阶段在独立的功能部件进行
- **处理器级流水线**：多个处理器（核）串行起来，每个处理器（核）完成其专门的任务。

■ 按完成的功能分类

- **单功能流水线**：只完成一种如乘法或浮点运算等，譬如Pentium有1条整数运算流水线，1条整数和浮点运算流水线。
- **多功能流水线**：流水线的各段可以进行不同的连接，在不同情况下可完成不同功能。
 - **静态流水线**：同一时间内，多功能结构只能按一种功能的连接方式工作。
 - **动态流水线**：同一时间内，可以有多种功能的连接方式同时工作。它允许一些段正在实现某些运算时，另一些段可以实现另一种运算。

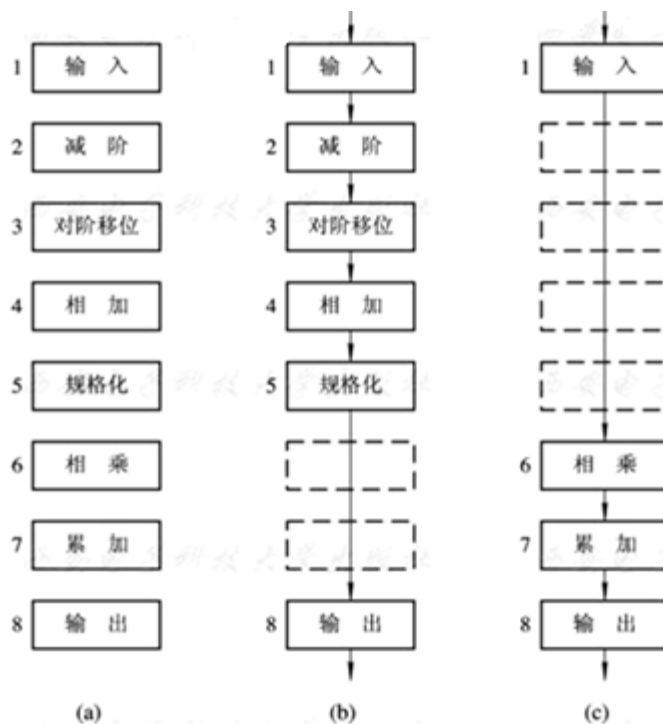
■ 按处理的数据类型分类

- **标量流水线**：每次操作处理一个数据元素
- **向量流水线**：通过向量指令对向量的各元素进行处理，每次操作处理多个相同类型的数据元素

■ 流水线结构上分类

- **线性流水线**：指各功能模块顺序串行连接，无反馈回路，
- **非线性流水线**：带有反馈回路的流水线，部件可能使用一次或多次。

◆ 多功能流水线



ASC机运算器的流水线

(a) 流水线的功能段；(b) 浮点加、减法运算时的连接；

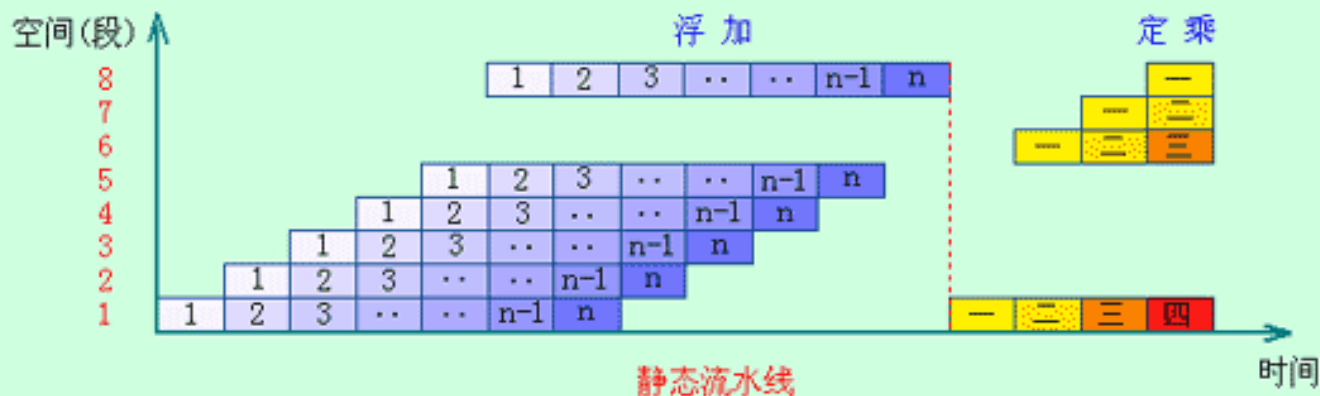
(c) 定点乘法运算时的连接

<https://blog.csdn.net/volvia12344321>

◆ 静、动态流水线时空图的对比

静、动态流水线的时空图

假设该流水线要先做几个浮点加法，然后再做一批定点乘法。



计算机性能评价（回顾）

❖ 响应时间与吞吐量

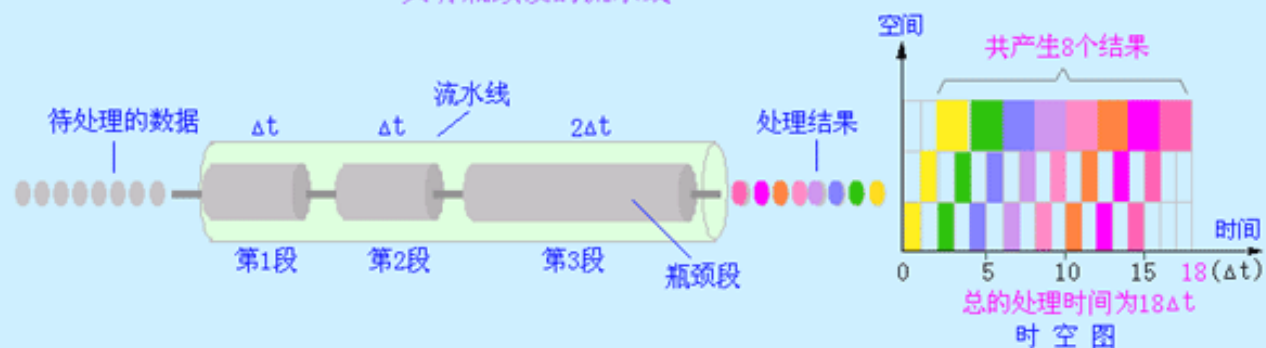
- **响应时间**：从提交作业到完成作业所花费的时间。
 - ◆ 响应时间是完成一个任务所花的时间总和，包括内存访问时间、执行IO操作的时间、以及运行必要的操作系统代码所需的时间。
- **吞吐量**：一定时间间隔内完成的作业数。
- 多任务操作系统更侧重于优化系统的整体吞吐量，而不会特别最小化某个特定程序的响应时间。
- 个人用户更关心响应时间，企业级计算机的管理人员更关心吞吐量。

流水线的性能指标-吞吐率

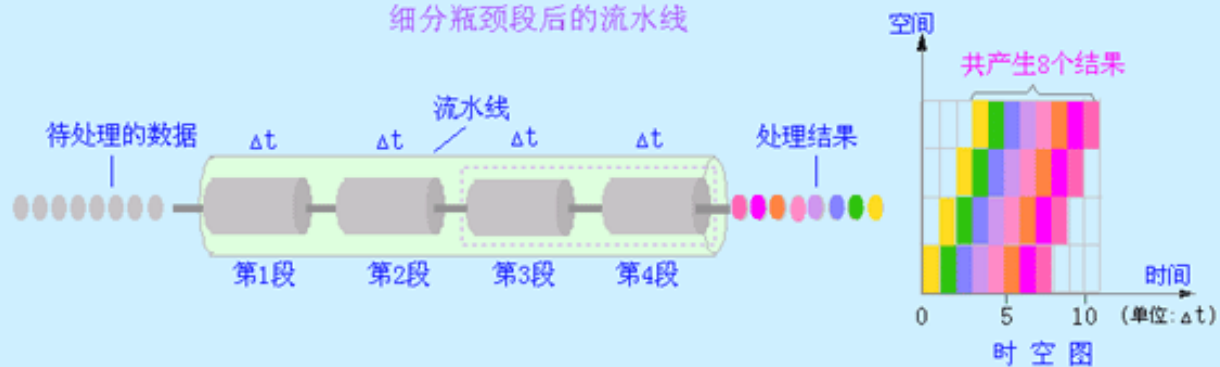
- ◆ **吞吐率**：单位时间内流水线所完成的任务数或输出结果的数量。
- ◆ **最大吞吐率** TP_{max} ：流水线在连续流动达到稳定状态后所得到的吞吐率。
 - ◆ 若流水线各段的时间相等，均为 Δt_0 ，则 $TP_{max} = 1 / \Delta t_0$ ，
即每过 Δt_0 时间，则有一条指令完成
 - ◆ 若流水线各段的时间不等，则： $TP_{max} = 1 / \max\{ \Delta t_0 \}$
 - ◆ 最大吞吐率取决于流水线中最慢的一段所需的时间，这段就成为流水线的瓶颈，消除瓶颈的方法：**细分瓶颈段**和**重复设置瓶颈段**

流水线的改进

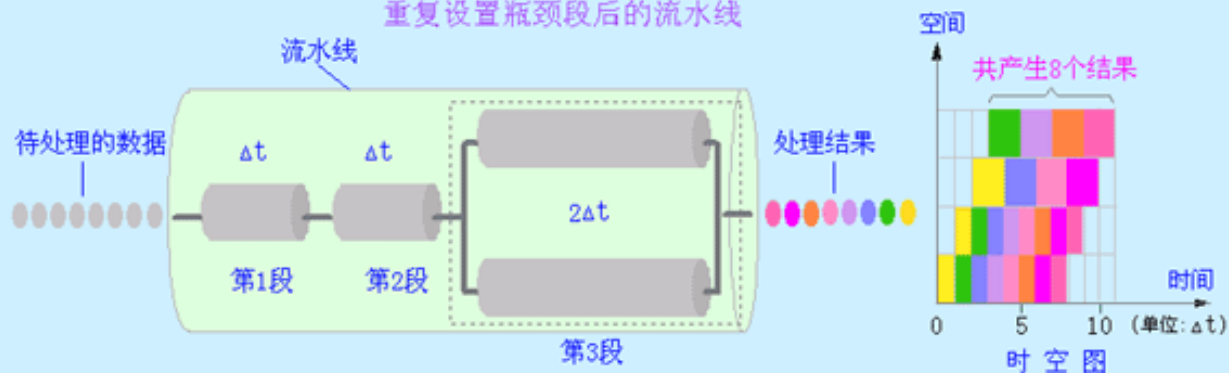
具有瓶颈段的流水线



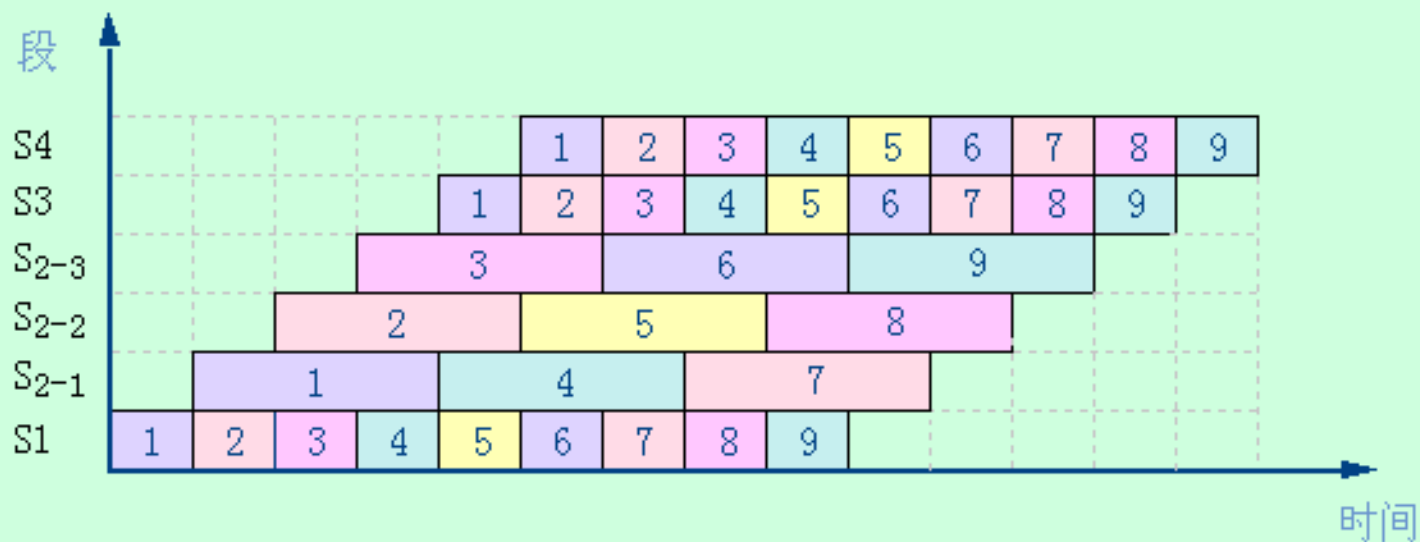
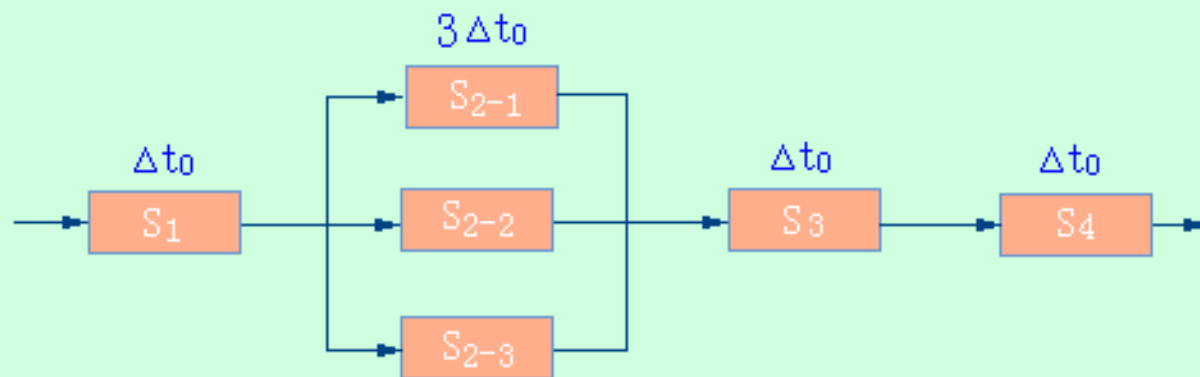
细分瓶颈段后的流水线



重复设置瓶颈段后的流水线



重复设置瓶颈段(时-空图举例)

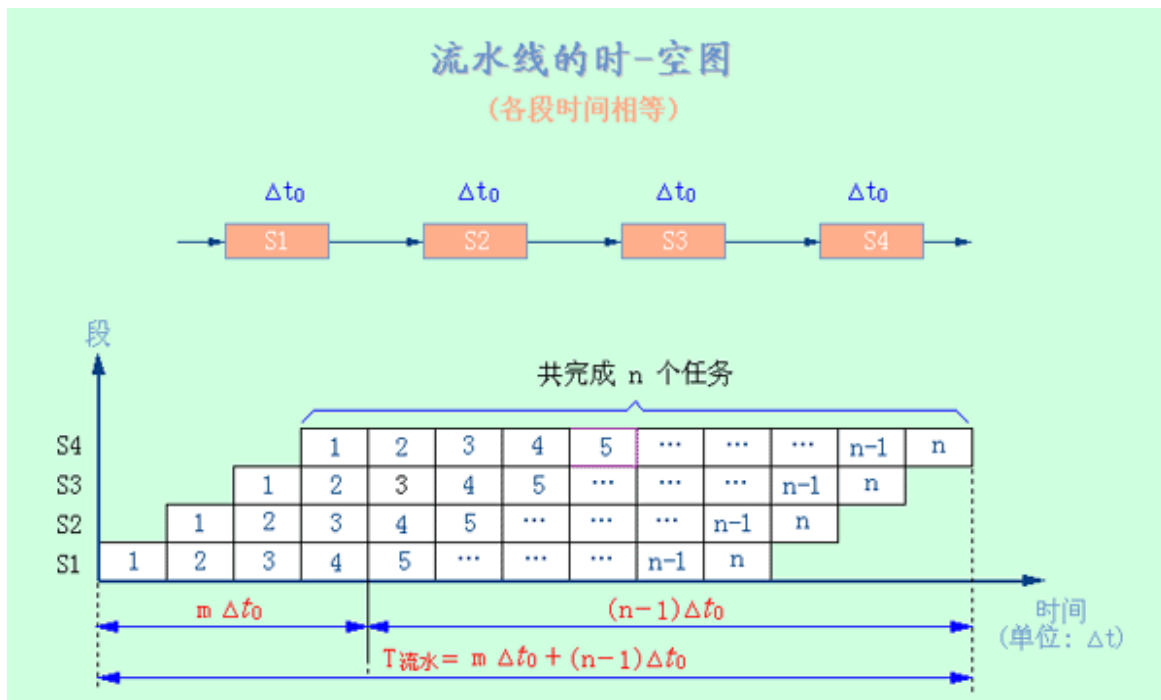


流水线的性能指标-吞吐率

◆ **实际吞吐率TP**：流水线的实际吞吐率小于最大吞吐率。

◆ 各段时间相等（设为 Δt_0 ），假设流水线由 m 段组成，完成 n 个任务所需的时间：

$$T_{\text{流水}} = m\Delta t_0 + (n - 1)\Delta t_0$$



实际吞吐率：

$$\begin{aligned} TP &= \frac{n}{T_{\text{流水}}} \\ &= \frac{n}{m\Delta t_0 + (n-1)\Delta t_0} \\ &= \frac{1}{\Delta t_0(m+n-1)/n} \\ &= \frac{TP_{\max}}{(m+n-1)/n} \\ &= \frac{TP_{\max}}{1+(m-1)/n} < TP_{\max} \end{aligned}$$

流水线的性能指标-加速比

◆ **加速比S**：流水线的速度与等功能非流水线的速度之比。

$$S = T_{\text{非流水}} / T_{\text{流水}}$$

$T_{\text{流水}}$ 和 $T_{\text{非流水}}$ 分别为按流水和按非流水方式处理 n 个任务所需的时间

若流水线为 m 段，且各段时间相等，均为 Δt_0 ，则：

$$T_{\text{非流水}} = n m \Delta t_0$$

$$T_{\text{流水}} = m \Delta t_0 + (n - 1) \Delta t_0$$

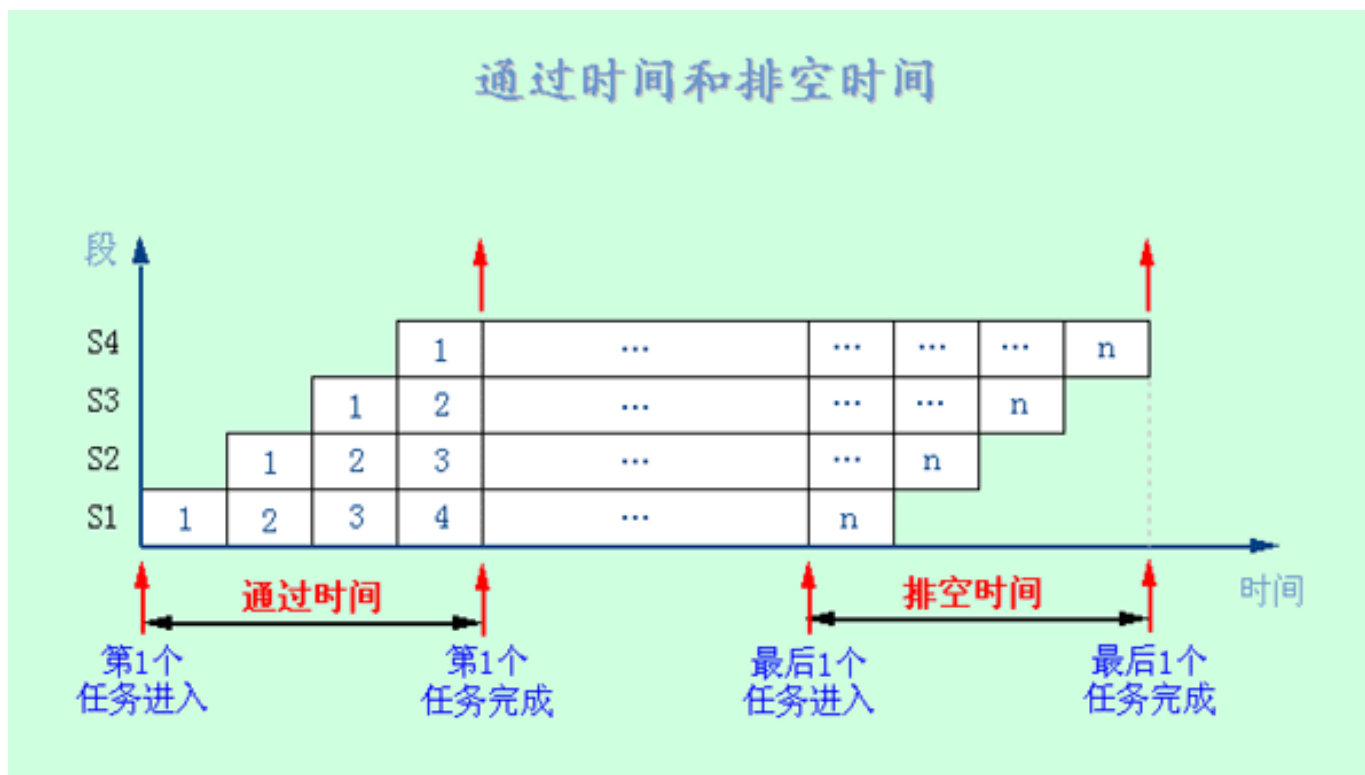
$$S = \frac{T_{\text{非流水}}}{T_{\text{流水}}} = \frac{nm\Delta t_0}{m\Delta t_0 + (n-1)\Delta t_0} = \frac{nm}{m+n-1}$$

可以看出：当 $n \gg m$ 时， $S \approx m$

流水线的性能指标-效率

◆ **效率 E:** 效率是指流水线的设备利用率

◆ 由于流水线有通过时间和排空时间, 所以流水线的各段并不是一直满负荷地工作。故: $E < 1$



流水线的性能指标-效率

- 若各段时间相等，则各段的效率 e_i 相等，即

$$e_1 = e_2 = e_3 = \dots = e_m = n \Delta t_0 / T_{\text{流水}}$$

整个流水线的效率为:

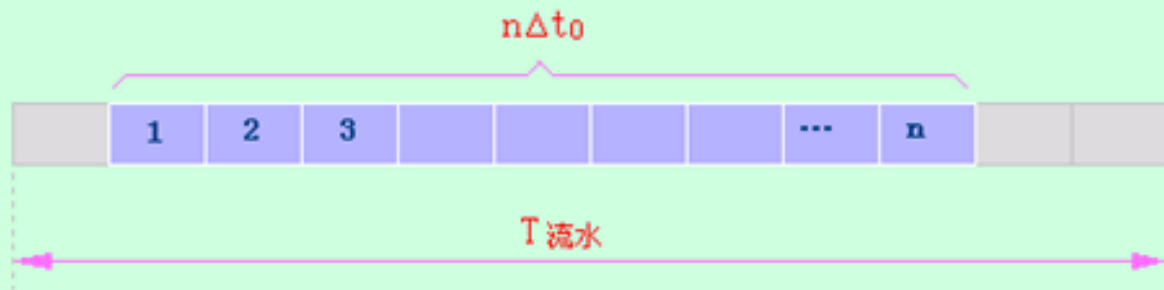
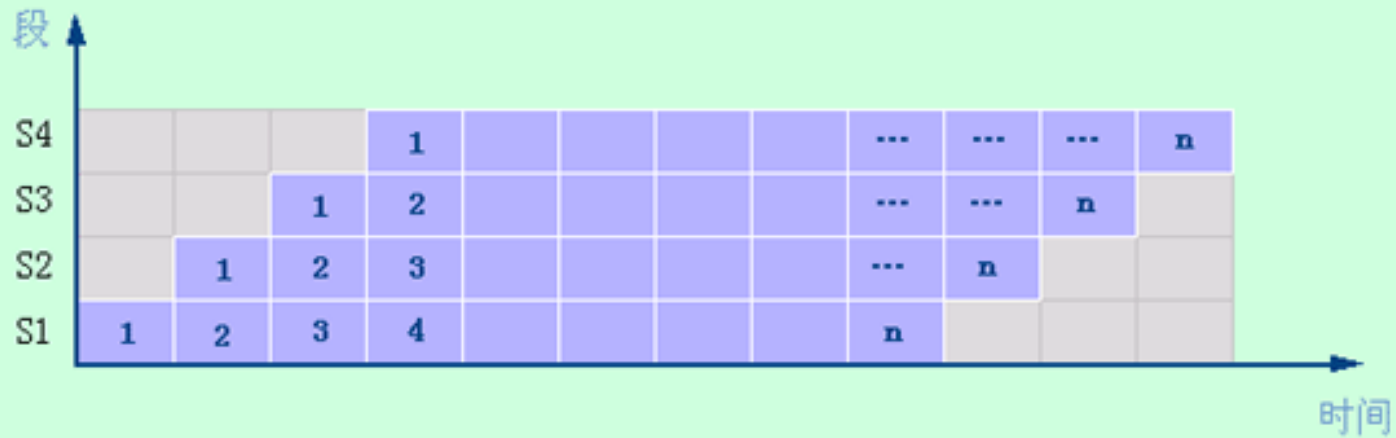
$$E = \frac{n \Delta t_0}{T_{\text{流水}}} = \frac{n}{n + m - 1}$$

当 $n \gg m$ 时, $E \approx 1$

- 从时空图上看，效率实际上就是 n 个任务所占的时空区与 m 个段总的时空区之比。

$$E = \frac{n \text{个任务所占的时空区}}{m \text{个段总的时空区}}$$

流水段的效率



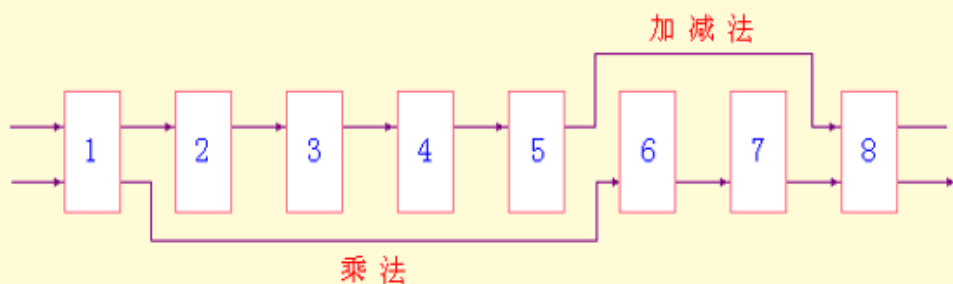
所以

$$e_2 = \frac{n\Delta t_0}{T_{\text{流水}}} (m\Delta t_0) + (n-1)\Delta t_0$$

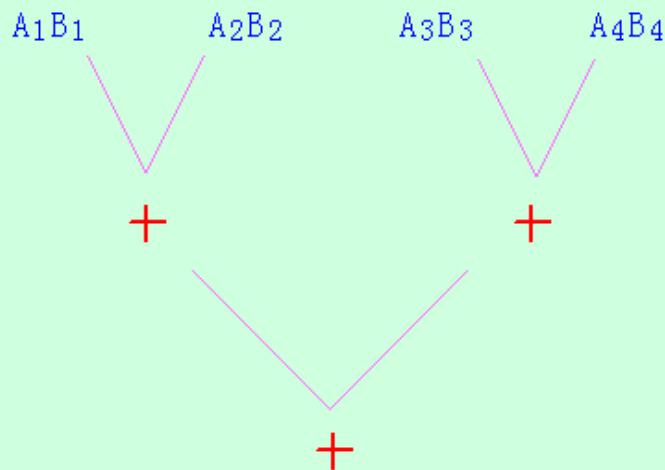
流水线性能分析举例

例 在静态流水线上计算 $\sum_{i=1}^4 A_i B_i$,
求：吞吐率，加速比，效率。

静态流水线
(举例)

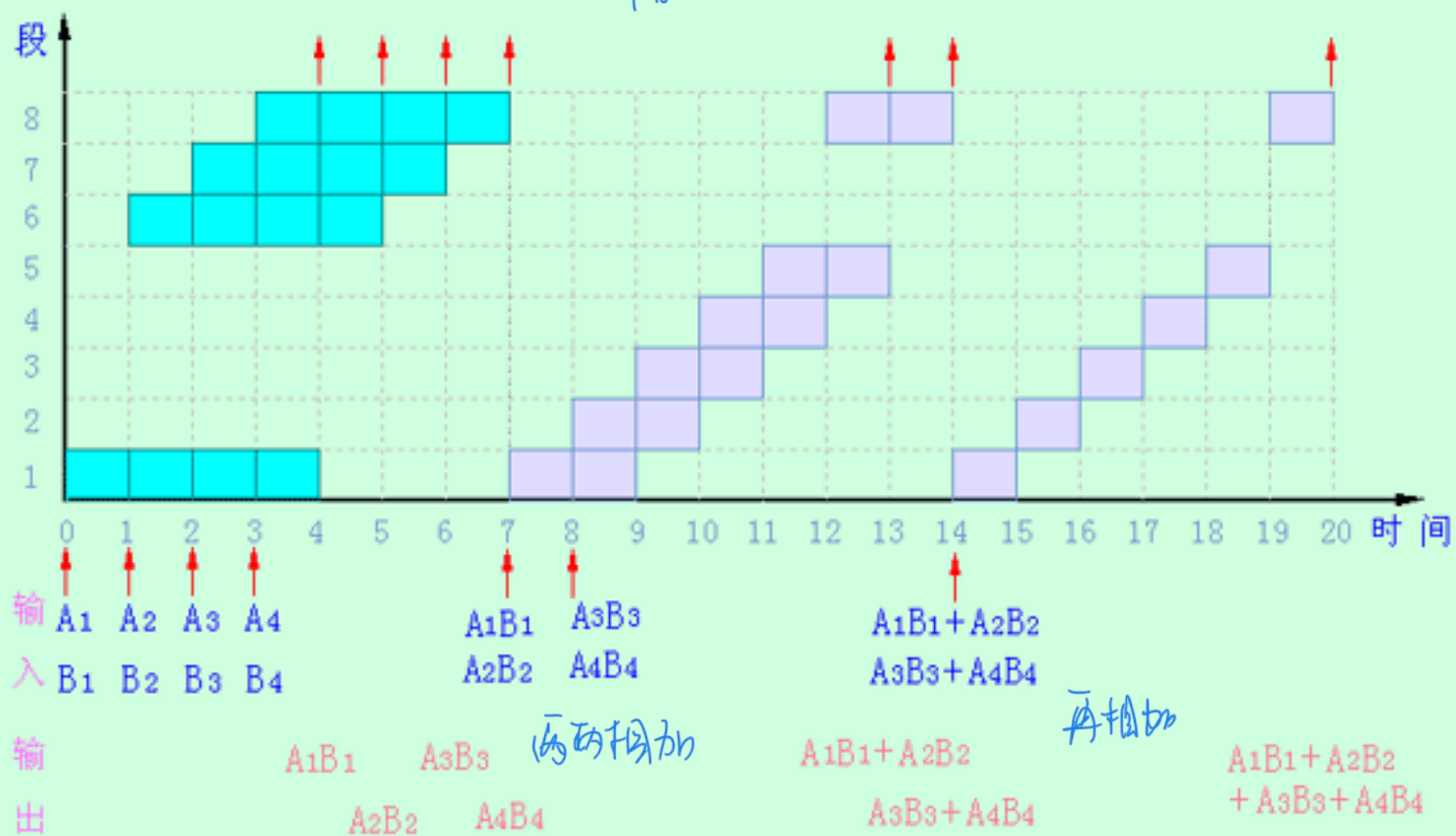


$$\sum_{i=1}^4 A_i B_i = A_1 B_1 + A_2 B_2 + A_3 B_3 + A_4 B_4$$



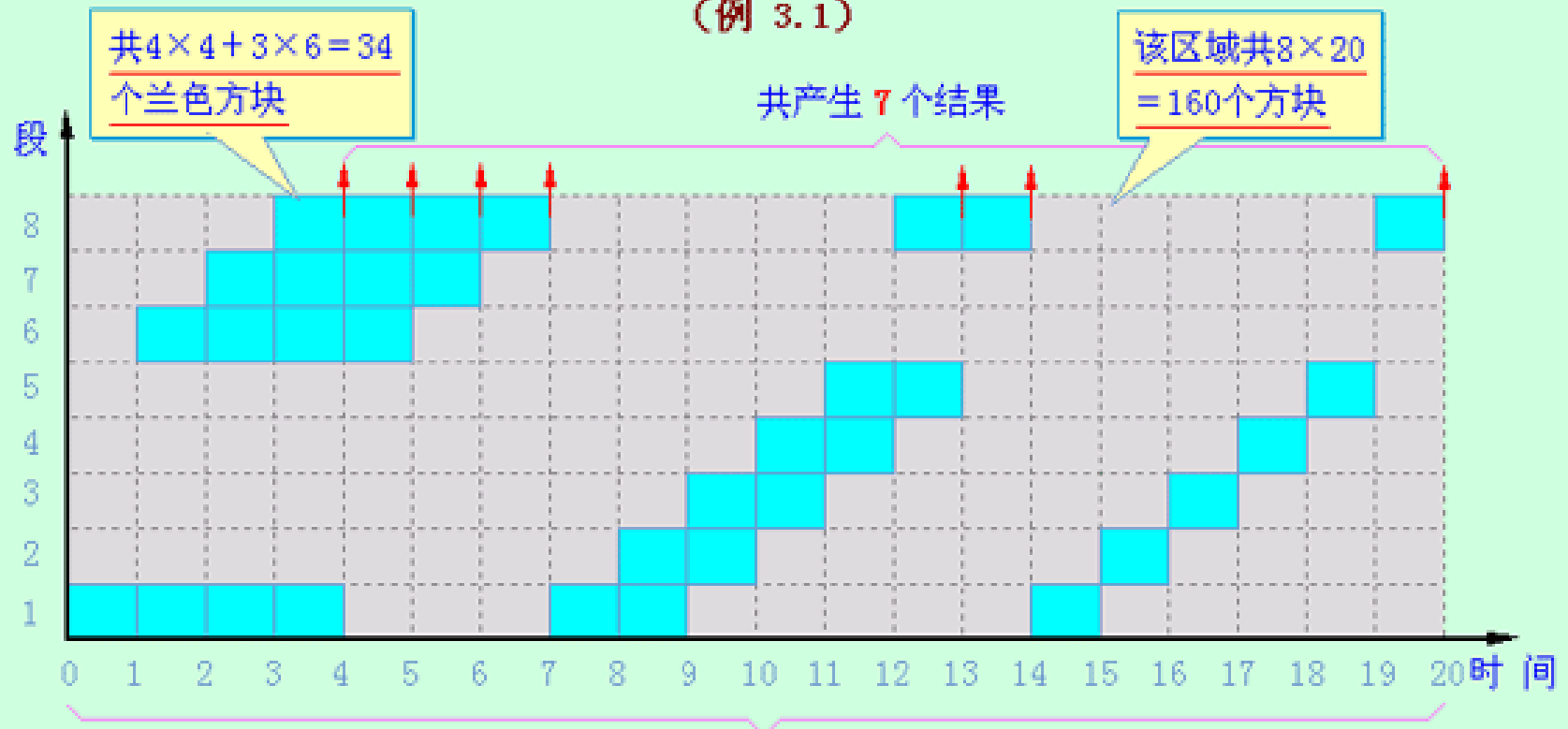
乘: 1678

+6: 122458



性能计算

(例 3.1)



吞吐率 $TP = \frac{7}{20 \Delta t}$ 处理时间 = $20 \Delta t$ 非流水处理所需时间 = $4 \times 4 \Delta t + 3 \times 6 \Delta t = 34 \Delta t$

加速比 $S = \frac{34 \Delta t}{20 \Delta t} = 1.7$ 效率 $E = \frac{34}{160} = 0.21$

主要内容

- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期流水线处理器设计
 - 4.6.1 多周期处理器设计概述
 - 4.6.2 流水线数据通路设计
 - 4.6.3 控制信号设计
- 4.7 流水线冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例

4.6.1 多周期处理器设计概述

■ 为什么不使用单周期实现方式

- 单周期设计中，取时钟周期对所有指令等长，由执行时间最长的指令决定，但某些指令类型本来可以在更短时间内完成。
- 单周期设计不能利用流水线重叠执行多条指令，效率低。

■ 多周期方案

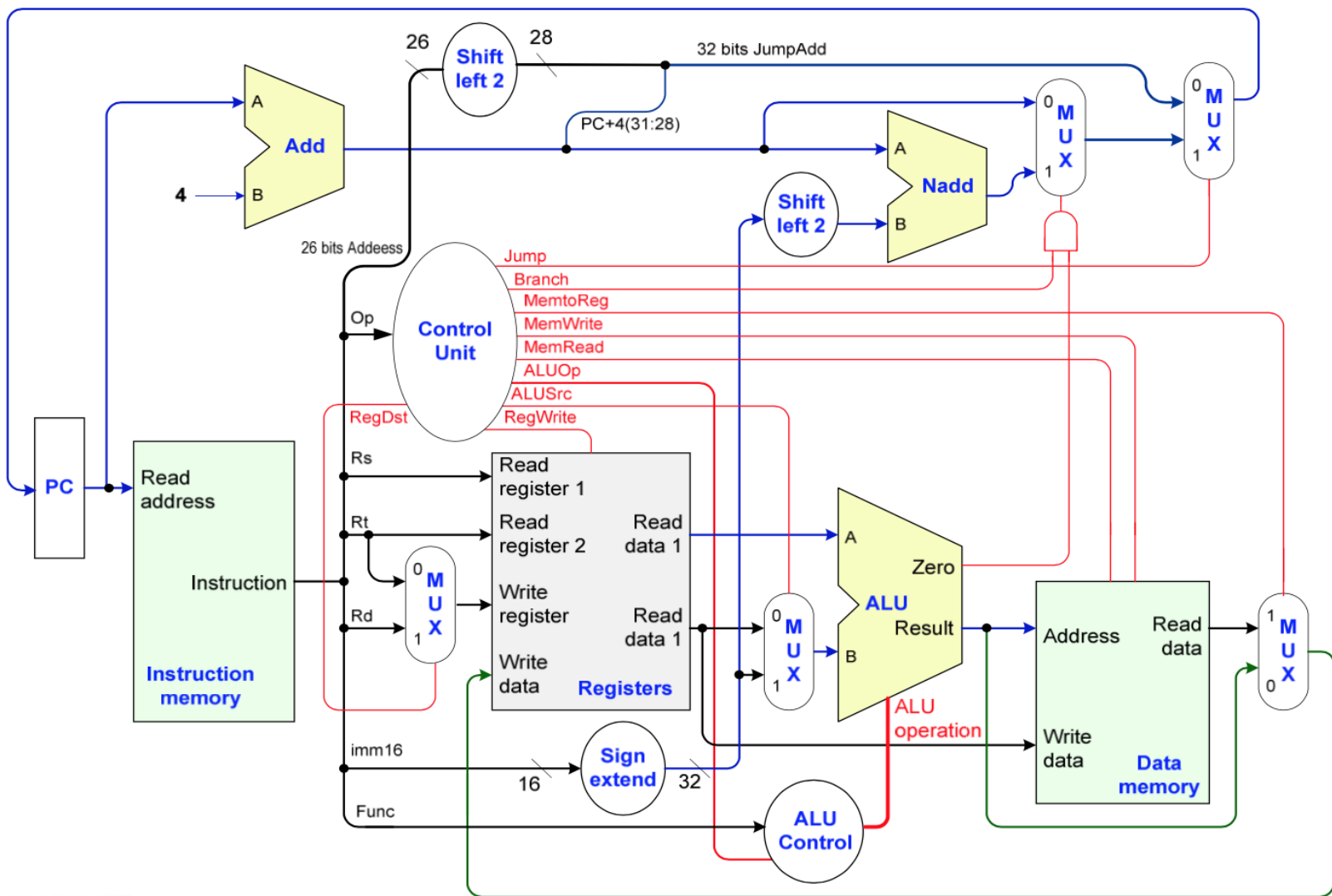
- 将指令执行分解为多个步骤，每个步骤一个时钟周期，则指令执行周期为多个时钟周期，不同指令的指令周期包含时钟周期数不一样。
- 优点：
 - 提高性能：不同指令的执行占用不同的时钟周期数；
 - 降低成本，提高硬件的利用率：一个功能单元可以在一条指令执行过程中使用多次；
- 缺点：
 - 增加非体系结构状态部件（寄存器），保存中间计算结果，增加延迟。
 - 控制逻辑更为复杂，需要有限状态机FSM而不是组合逻辑。

单周期CPU设计回顾

- 支持指令类型
 - ALU计算型 (R型)
 - 访存型指令 (I型)
 - 条件跳转指令 (I型)
 - 无条件跳转指令 (J型)
- 主要功能部件
 - PC、IM、DM、GR/Reg
 - ALU、Adder、Ext、Shift
- 时钟控制方式
 - 单周期 (足够长) *lw*
 - 周期结束的下降沿输入寄存器/DM

单周期CPU设计回顾

8条指令数据通路, 需要什么控制信号



MIPS多周期处理器设计

■ 多周期数据通路设计总体考虑

- **时钟同步**方法：一个时钟周期内信号总是从一个状态单元经过组合逻辑处理后传送到另一个状态单元。
- 指令每一步的执行总是从前一个状态单元接收输入，经过功能单元处理，在下一个时钟周期触发沿将结果写入下一个状态单元。
- 数据通路中需要增加了一个或多个寄存器以保存指令各执行步骤形成的结果（输出值），以便在指令的后续时钟周期内继续使用。

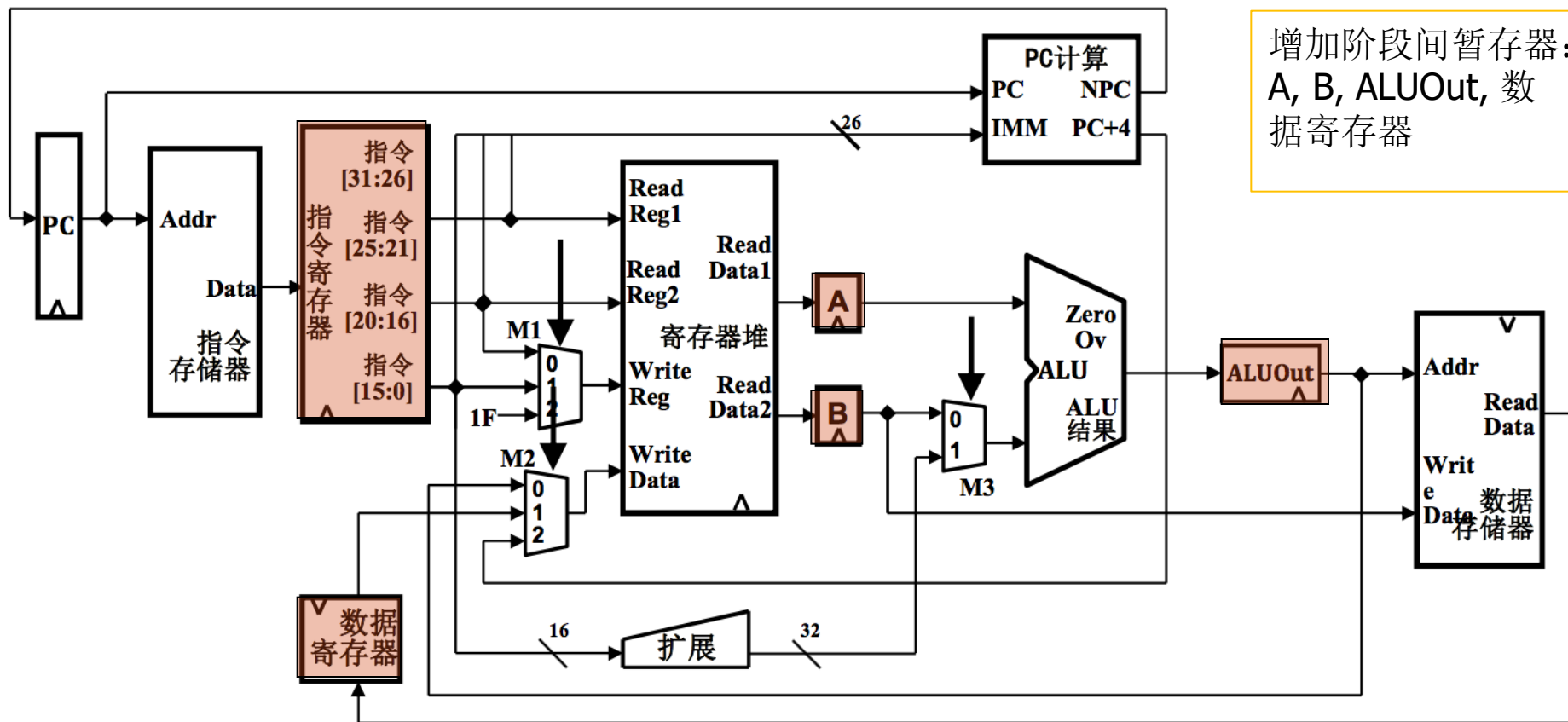


- 组合逻辑操作时钟周期内完成;
- 信号在时钟周期内从状态单元1经组合逻辑传送到状态单元2。
- 时钟信号上跳沿同步

数据通路的五个阶段

- ① 取指-IF：取指, PC自增
- ② 译码和读寄存器-ID：译码, 读寄存器
- ③ 执行-EX：执行计算(ALU)
 - Lw/Sw指令：计算内存地址
 - 其他指令：执行算术和逻辑运算
- ④ 访存-Mem：
 - Lw：从内存读数据到CPU寄存器
 - Sw：把寄存器的值写到内存中
- ⑤ 回写-WB：把数据写回到寄存器中 *R/lw*

多周期处理器设计（非流水）



多周期性能分析

图写, 要准备数据, 不是写寄存器



■ 假设主要功能单元的操作时间

- 存储器 : 200ps
- ALU: 100ps
- 寄存器堆: 100ps *读寄存器: 100ps*
- 数据选择器、控制单元、PC、符号扩展单元、线路没有延迟

写寄存器. Setup

各类指令执行时间

步骤	R型指令	Lw指令	Sw指令	Beq指令	J指令	执行时间
取指令	$IR \leftarrow M[PC], PC \leftarrow PC + 4$					200ps
读寄存器/ 译码	$A \leftarrow R[IR[25:21]], B \leftarrow R[IR[20:16]]$ $AdderOut \leftarrow PC + Signext[IR[15:0]] \ll 2$				$PC \leftarrow PC[31:28]$ $\parallel IR[25:0] \ll 2$	100ps
计算 <i>(350ps)</i>	$ALUOut \leftarrow A \text{ op } B$	$ALUOut \leftarrow A + Signext(IR[15:0])$		If $(A-B == 0)$ then $PC \leftarrow ALUOut$		100ps
R型完成/ 访问内存	$R[IR[15:11]] \leftarrow ALUOut$	$DR \leftarrow M[ALUOut]$	$M[ALUOut] \leftarrow B$			<i>图写 访存</i> 100/200ps
写寄存器		$R[IR[20:16]] \leftarrow DR$				100ps

多周期性能分析

❖ 时钟周期：取各步骤中最长的时间，200ps

步骤	R型指令	Lw指令	Sw指令	Beq指令	J指令	执行时间
TC1	IR ← M[PC], PC ← PC + 4					200ps
TC2	A ← R[IR[25:21]], B ← R[IR[20:16]] AdderOut ← PC + Signext[IR[15:0]] << 2				PC ← PC[31:28] IR[25:0] << 2	200ps
TC3	ALUOut ← A op B	ALUOut ← A + Signext(IR[15:0])		If (A-B == 0) then PC ← ALUOut		200ps
TC4	R[IR[15:11]] ← ALUOut	DR ← M[ALUOut]	M[ALUOut] ← B			200ps
TC5		R[IR[20:16]] ← DR				200ps

多周期性能分析

❖ 各型指令所需的时钟周期数和时间

- R型指令：4, 800ps
- lw指令：5, 1000ps
- sw指令：4, 800ps
- beq指令：3, 600ps
- j指令：2, 400ps

❖ 假设指令在程序中出现的频率

- lw指令：25%
- sw指令：10%
- R型指令：45%
- beq指令：15%
- j指令：5%

❖ 则一条指令的平均CPI

cycles per instruction

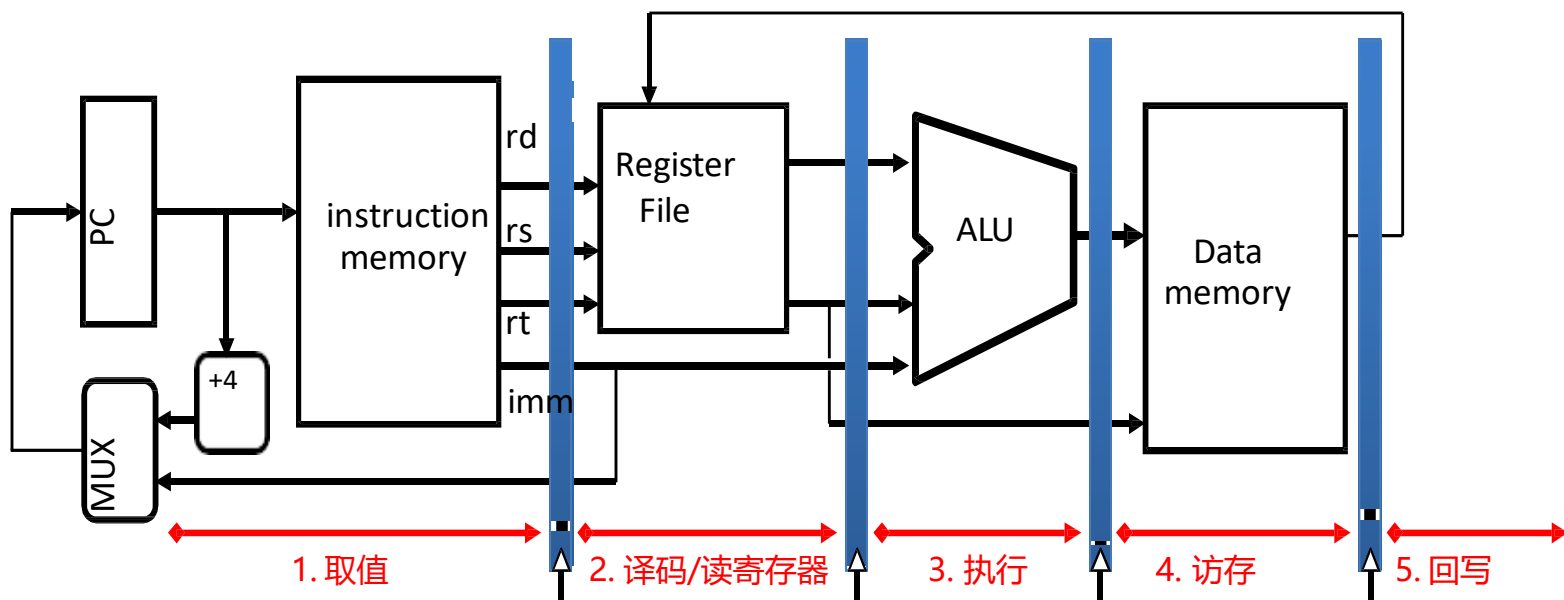
- $5 \times 25\% + 4 \times 10\% + 4 \times 45\% + 3 \times 15\% + 2 \times 5\% = 4$

4.6.2 流水线数据通路设计

■ 数据通路流水化:

段间寄存器

- 在两个阶段之间增加寄存器：保存前一时钟周期产生的数据
- 5 阶段流水线：理论上，时钟频率可以增加到原来的5倍



2.5 利用 Register File

流水线的优势

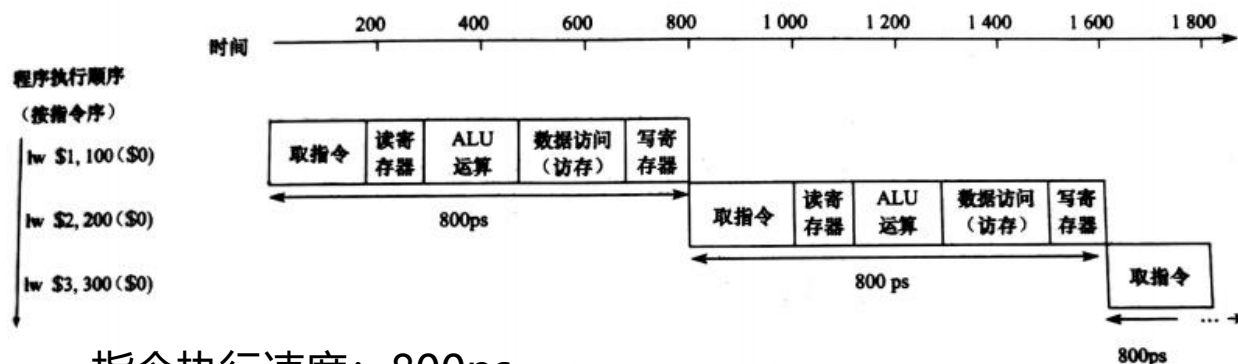
- 假设
 - 译码/读寄存器阶段和写寄存器阶段的花费为100ps
 - 其他阶段为200ps

指令	取指	读寄存器	运算执行	访问内存	写寄存器	总计
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

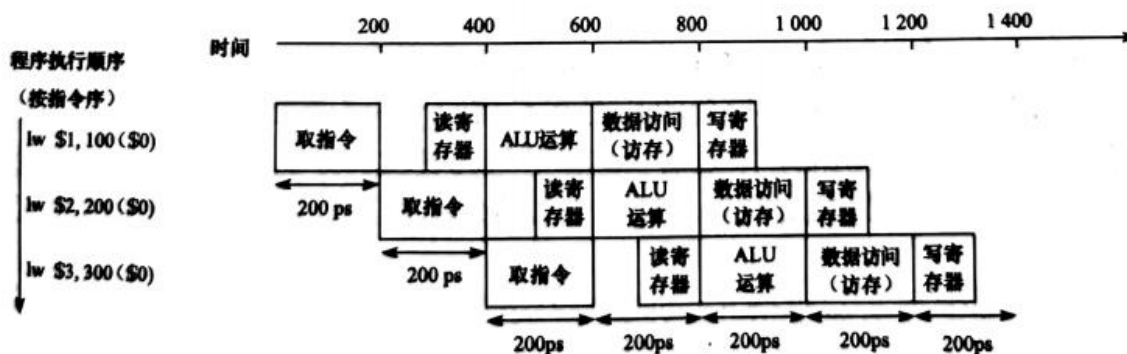
流水线的优势

- 单/多周期 vs. 多周期流水线

拆出来送最长的



指令执行速度: 800ps



每个周期发一个

指令执行速度: 200ps

思考练习：

4.8 本练习题讨论流水线对处理器时钟周期的影响。下面给出了数据通路中不同阶段的延迟情况：

IF	ID	EX	MEM	WB
250ps	350ps	150ps	300ps	200ps

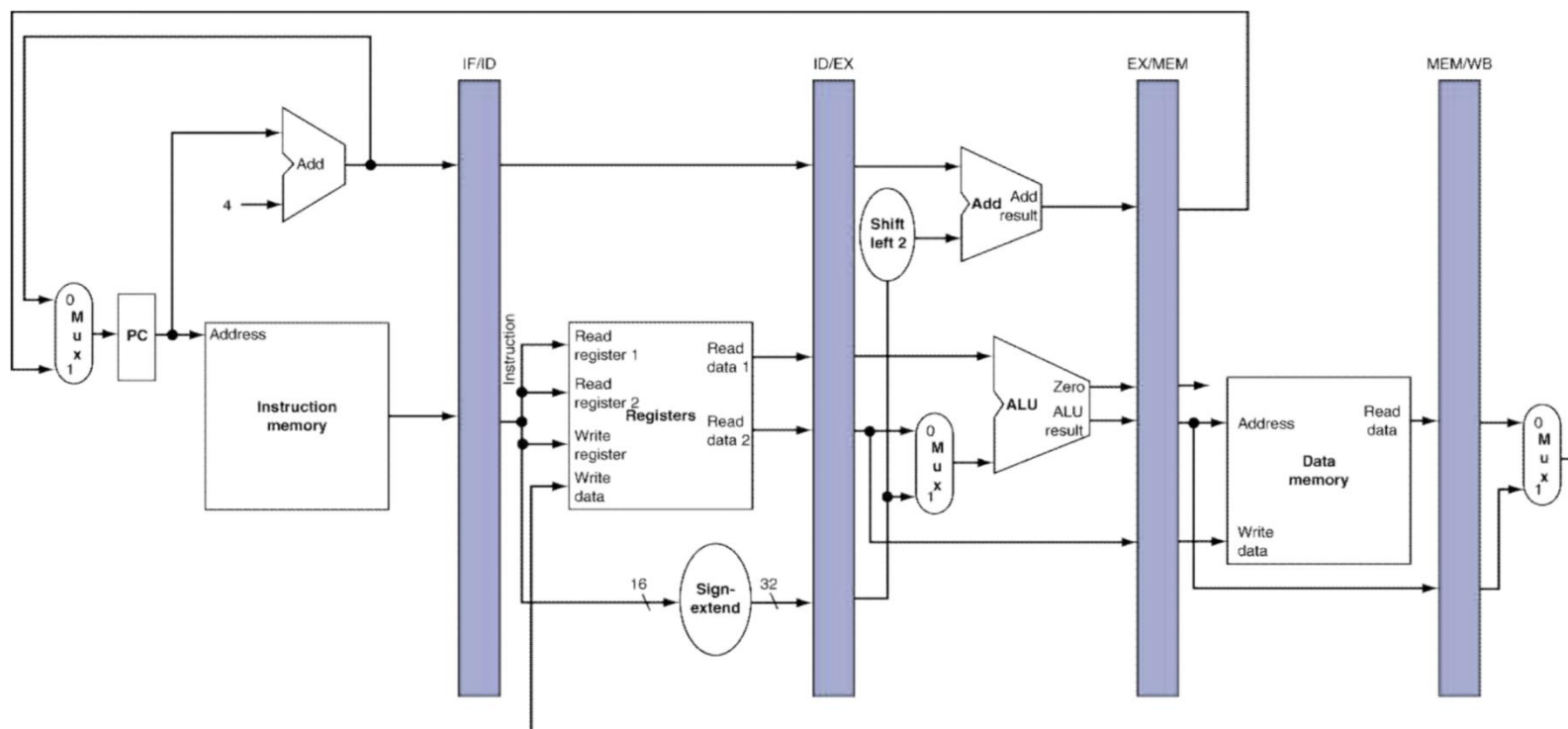
另外，假定处理器执行各种指令的比率如下面所示：

alu	beq	lw	sw
45%	20%	20%	15%

- 4.8.1 [5] <4.5> 流水线处理器与非流水线处理器的时钟周期分别是多少？
- 4.8.2 [10] <4.5> lw指令在流水线处理器和非流水线处理器中的总延迟分别是多少？
- 4.8.3 [10] <4.5> 如果可以将原流水线数据通路的一级划分为两级，每级的延迟是原级的一半，那么你会选择哪一级进行划分？划分后处理器的时钟周期为多少？
- 4.8.4 [10] <4.5> 假设没有阻塞和冒险，数据存储器的利用率是多少（占总周期数的百分比）？
- 4.8.5 [10] <4.5> 假设没有阻塞和冒险，寄存器堆的写寄存器端口的利用率是多少？
- 4.8.6 [30] <4.5> 假设一种多周期的处理器设计，其中每条指令需要多个时钟周期完成，但上一条指令完成前不取下一条指令。在这种设计中，指令仅经过其所需的阶段（例如，存储指令仅需4个时钟周期，因为其不需要WB阶段）。比较单周期设计、多周期设计和流水线设计三者的时钟周期和总执行时间。

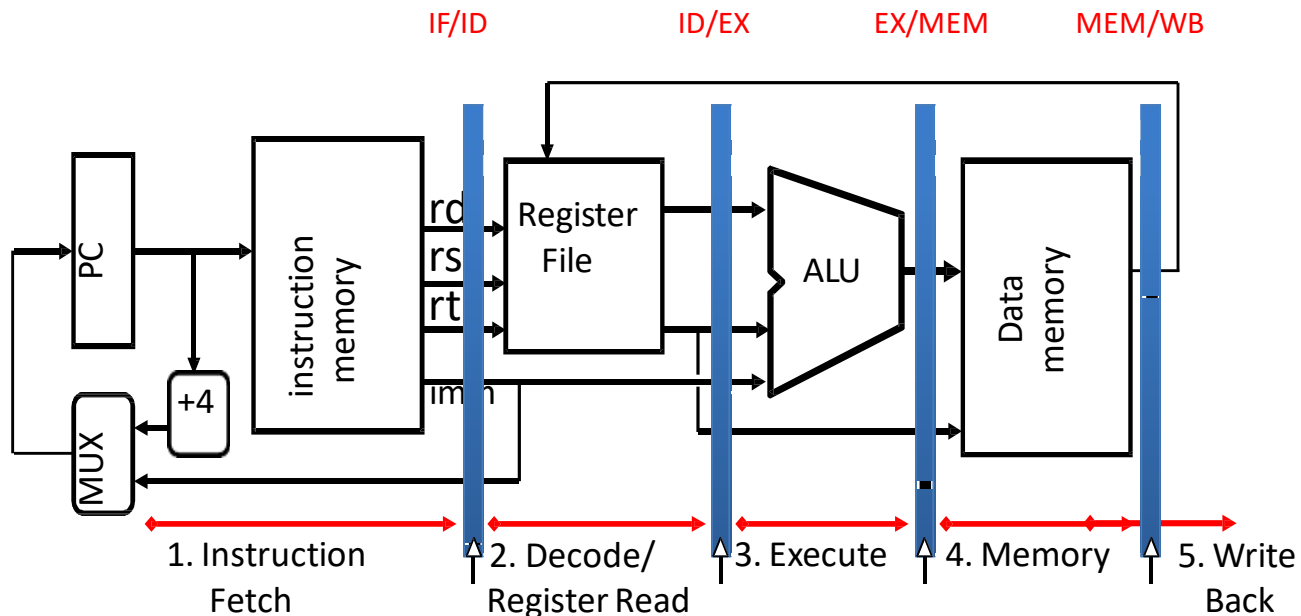
流水线数据通路设计

- 流水线模型指令执行步骤分解
 - E.g., 指令lw的流水线（5段都使用）



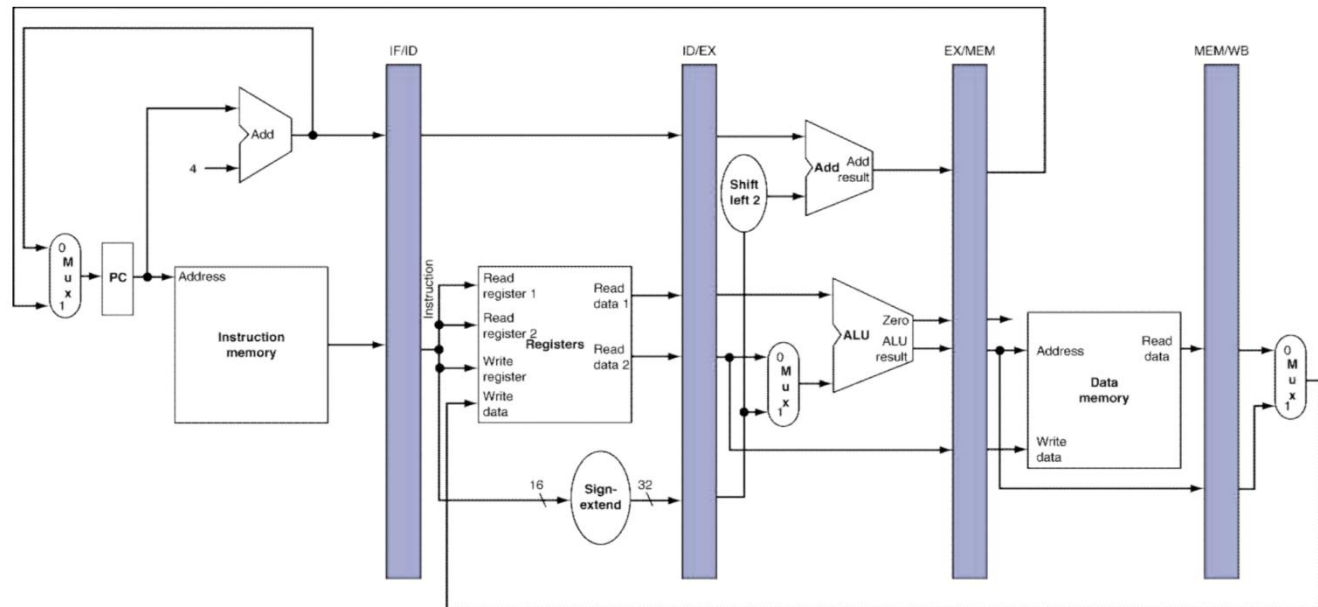
流水线寄存器

- 命名法则：前级/后级
 - 示例：IF/ID，前级为读取指令，后级为指令译码(及读操作数)
- 功能：时钟上升沿到来时，读取前级结果，然后输出至下级组合逻辑，也可能直接连接到下级流水线寄存器
 - 例如SW：ID/EX保存的从RF读出的第2个寄存器值，就直接传递到EX/MEM



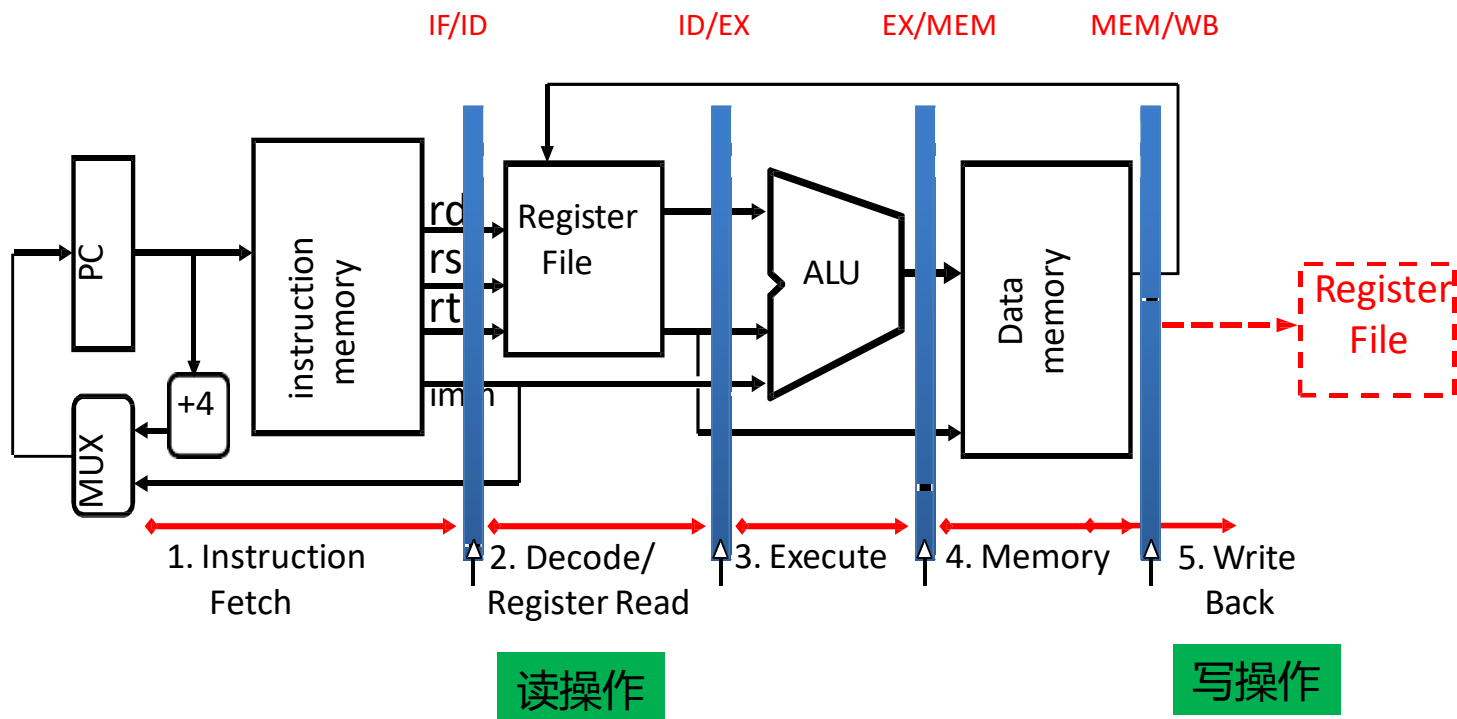
数据通路设计

- 各阶段间流水线寄存器位数 (初始设计)
 - IF/ID: 64位 (PC + IR)
 - ID/EX: 128位 (PC, A, B, Ext32)
 - EX/MEM: 97位 (PC, ALUOut, Flag, B)
 - MEM/WB: 64位 (Data, ALUOut)



流水线级数与RF

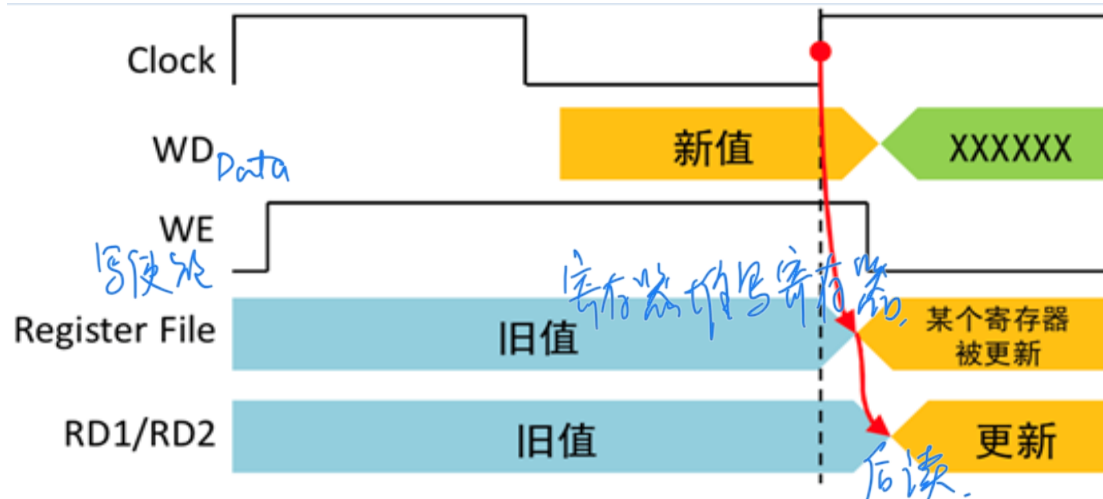
- N级流水线：应有N级流水线寄存器
 - 新增N-1级流水线寄存器
- RF：一个特殊部件，有2次使用
 - 读：第2级；写：第5级



先写后读，保证能读到新值

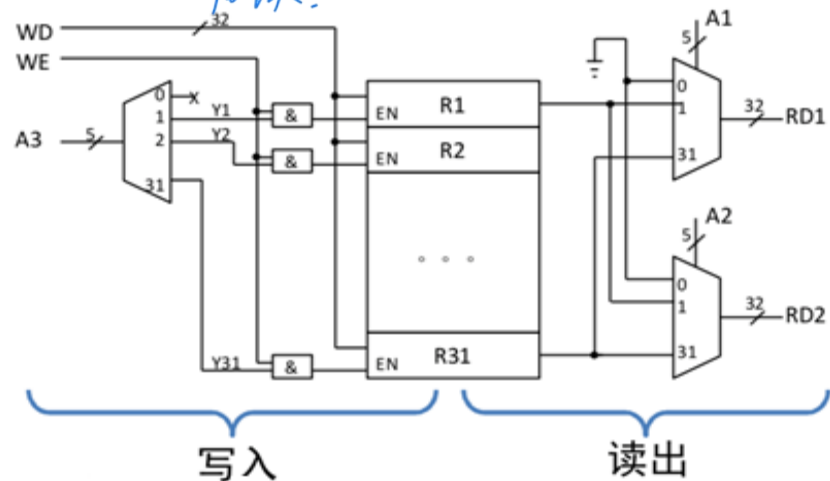
读写RF的时序

2F | WB



- RF具有2重行为
- 读出：组合逻辑
- 写入：时序逻辑(寄存器)

- RF：上升沿被更新
 - WE有效，则WD被写入A3
- 输出：与时钟无关，仅与RF内容及A1/A2相关



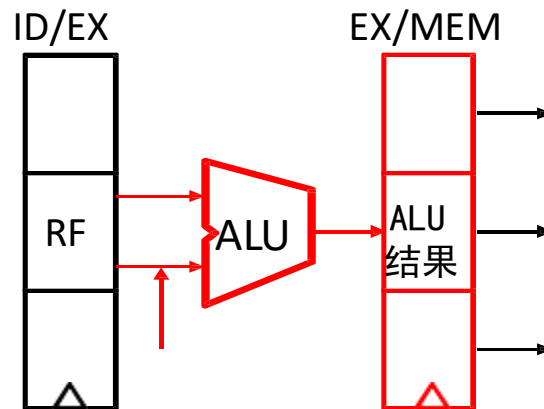
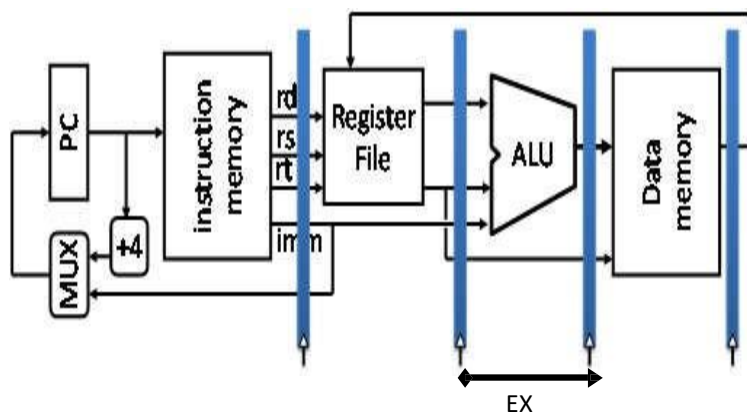
流水阶段与流水线寄存器

■ 流水阶段：寄存器+组合逻辑

- ◆ 起始：前级流水线寄存器的输出
- ◆ 中间：组合逻辑（如ALU）
- ◆ 结束：当时钟上升沿到来时，组合逻辑计算结果写入后级流水线寄存器

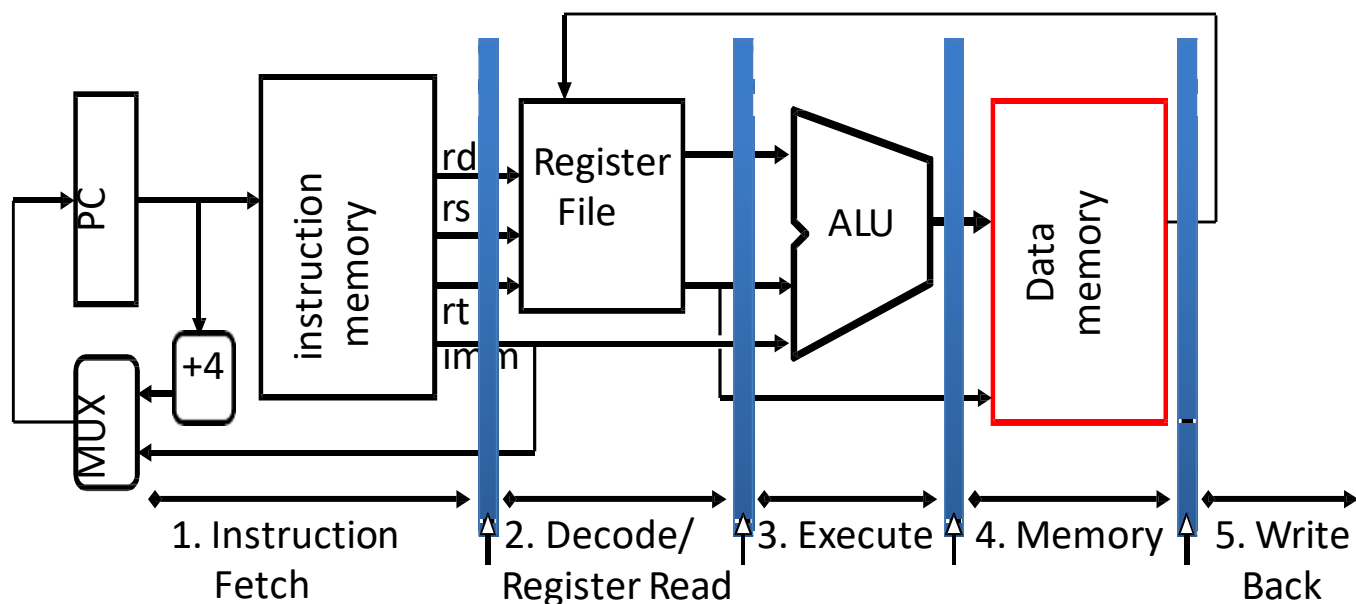
■ 示例：EX阶段

- ◆ 起始：ID/EX流水线寄存器中的RF寄存器/扩展单元的输出
- ◆ 中间(组合逻辑)：ALU完成计算
- ◆ 结束(寄存器)：在clock上升沿到来时，结果写入EX/MEM中相应寄存器



流水线部件：DM

- 写入时：表现为寄存器，属于MEM/WB寄存器范畴
- 读出时：组合逻辑
 - ◆ 与RF的读出是类似的，但需要相应的控制信号

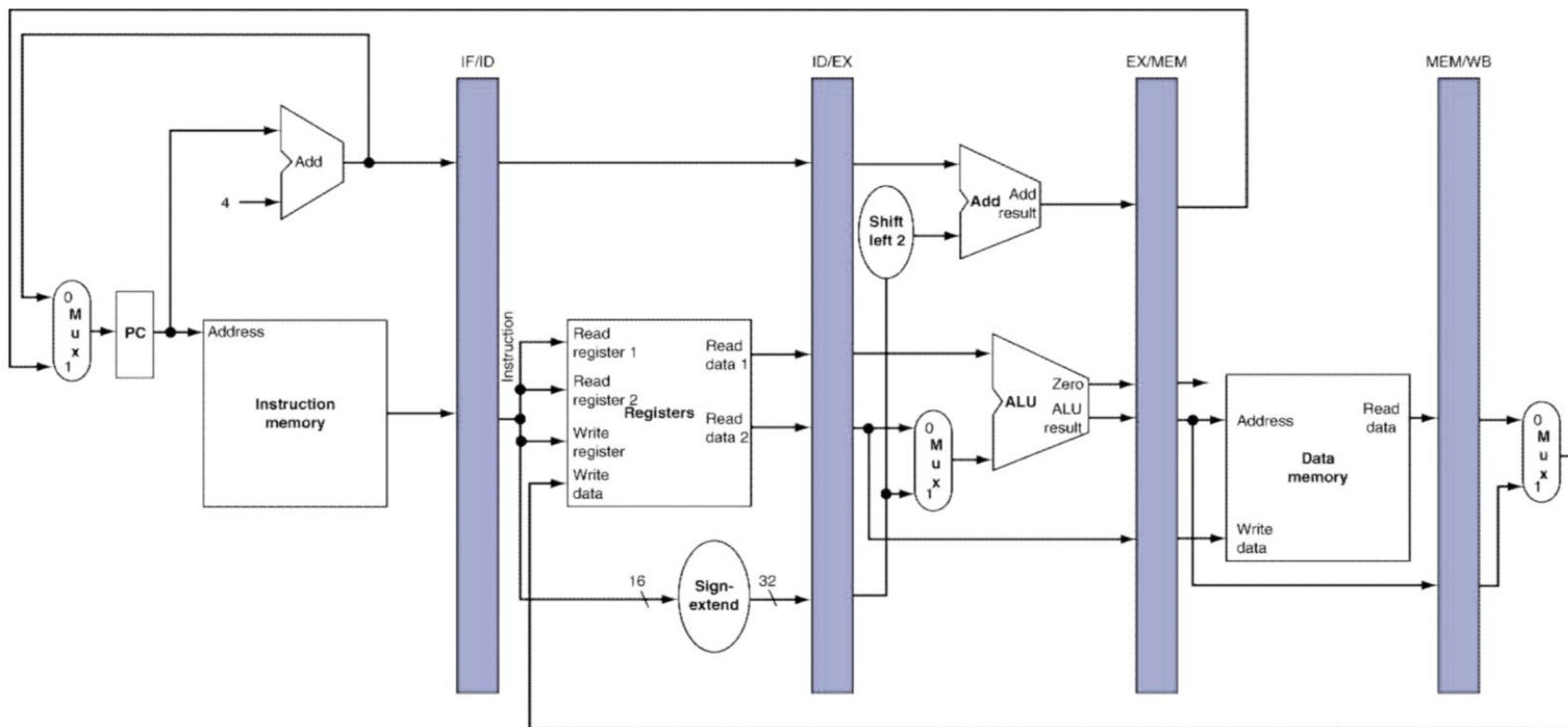


流水线寄存器与数据通路

- 寄存器的加入将会影响流水线上的信息
 - 为相邻的流水阶段增加寄存器 (e.g. IF/ID)
 - 寄存器可以将不同阶段的信息分隔开
 - 在任何时间，每个阶段都在执行不同的指令
- 在这种情况下需要再次检验数据通路

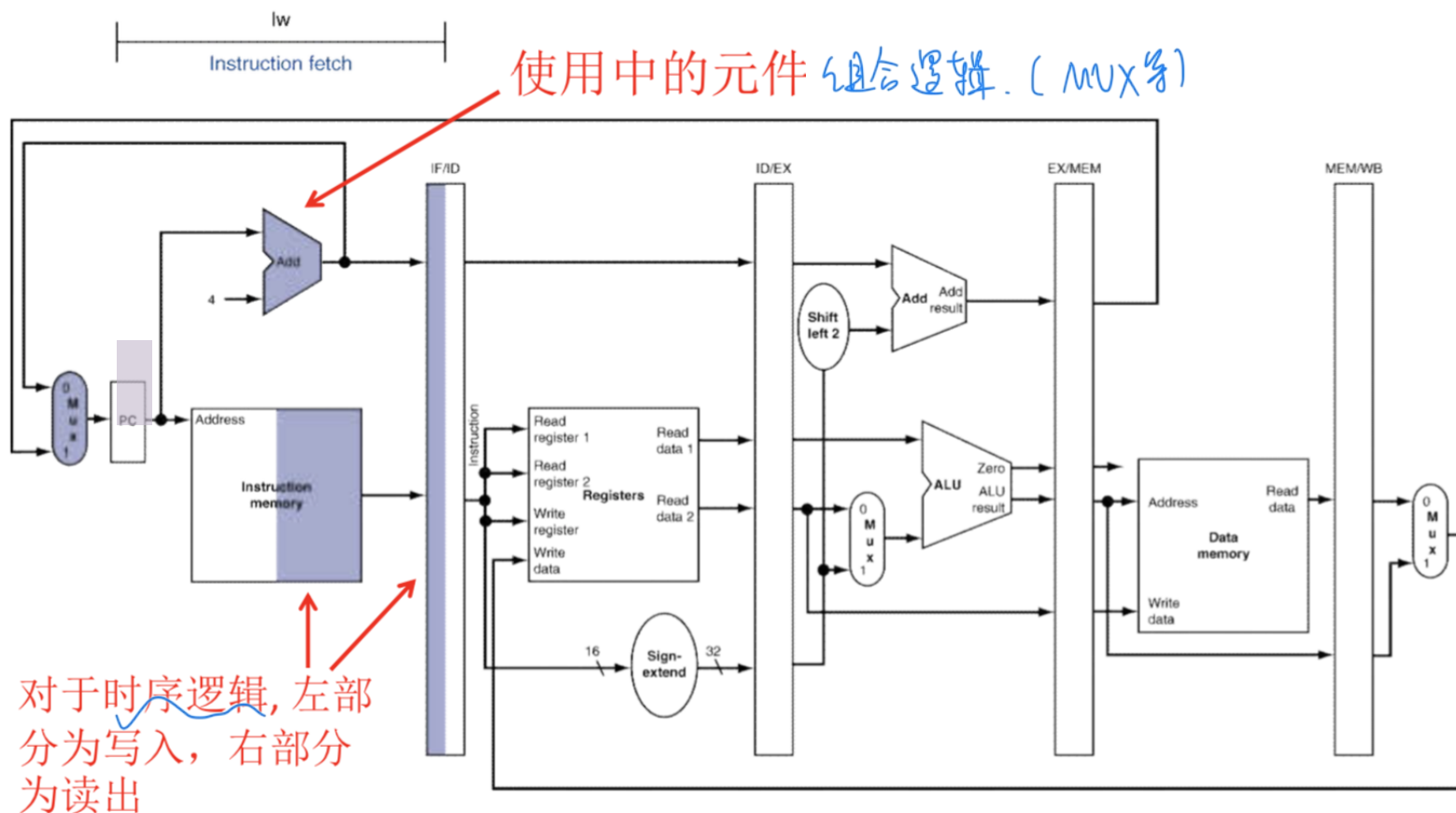
流水线寄存器

- 各阶段间寄存器
 - PC可以看做是一个流水线寄存器，给IF级提供数据。
 - 相对的，PC是可见结构，发生异常时必须保存
 - 其他流水线寄存器是不可见结构，异常时可以丢弃



流水线数据通路

- 指令lw的流水线：取指阶段
 - （存储器件右边阴影表示读，左边阴影表示写）
 - IF/ID: 64位 (PC + IR)



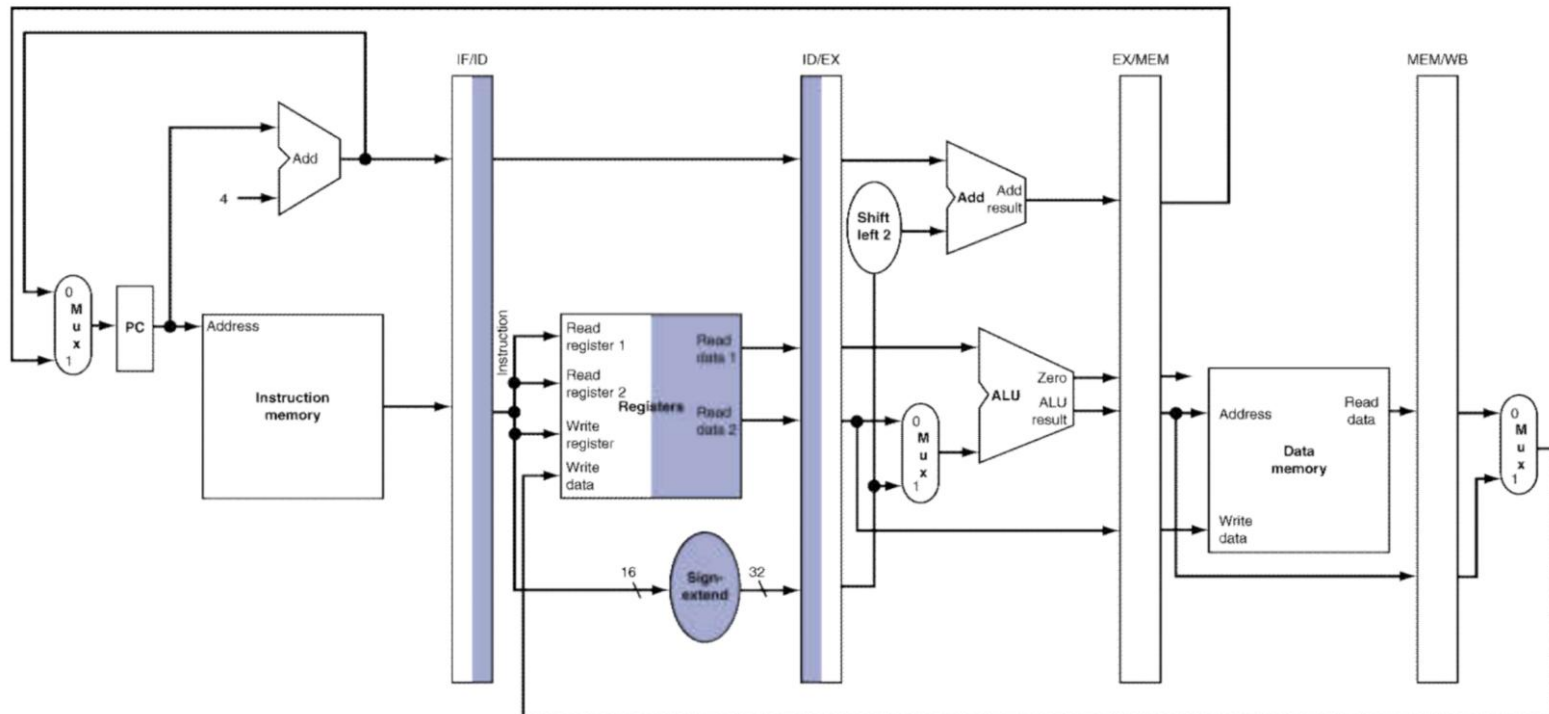
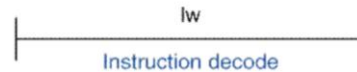
流水线数据通路

- 指令lw的流水线：译码阶段

- A, B, Ext32, 每次一般只需要用2个, ID/EX寄存器三个数都保存, 能简化控制

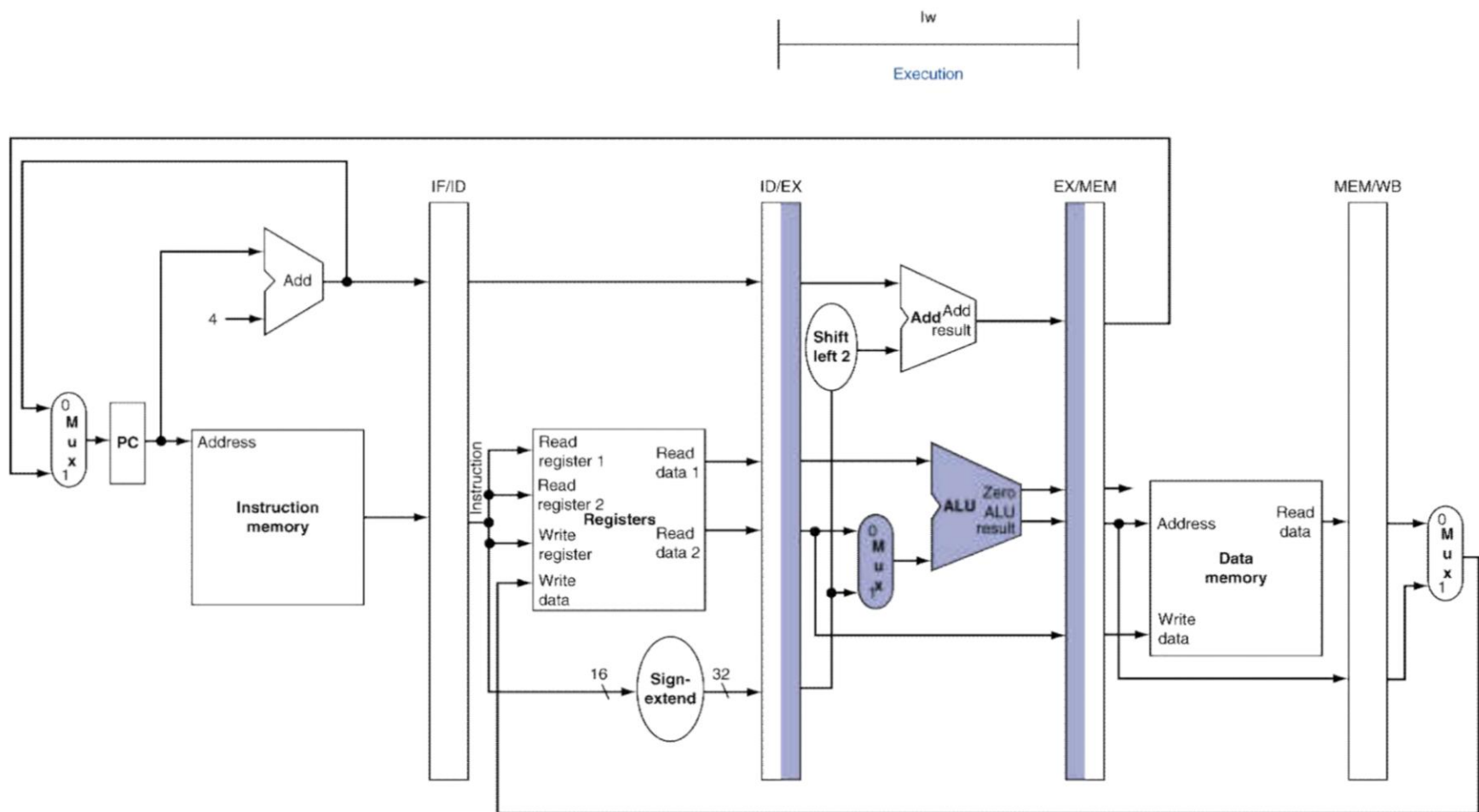
- ID/EX: 128位 (PC, A, B, Ext32) 4个32位供后续使用

译码, 读通用寄存器



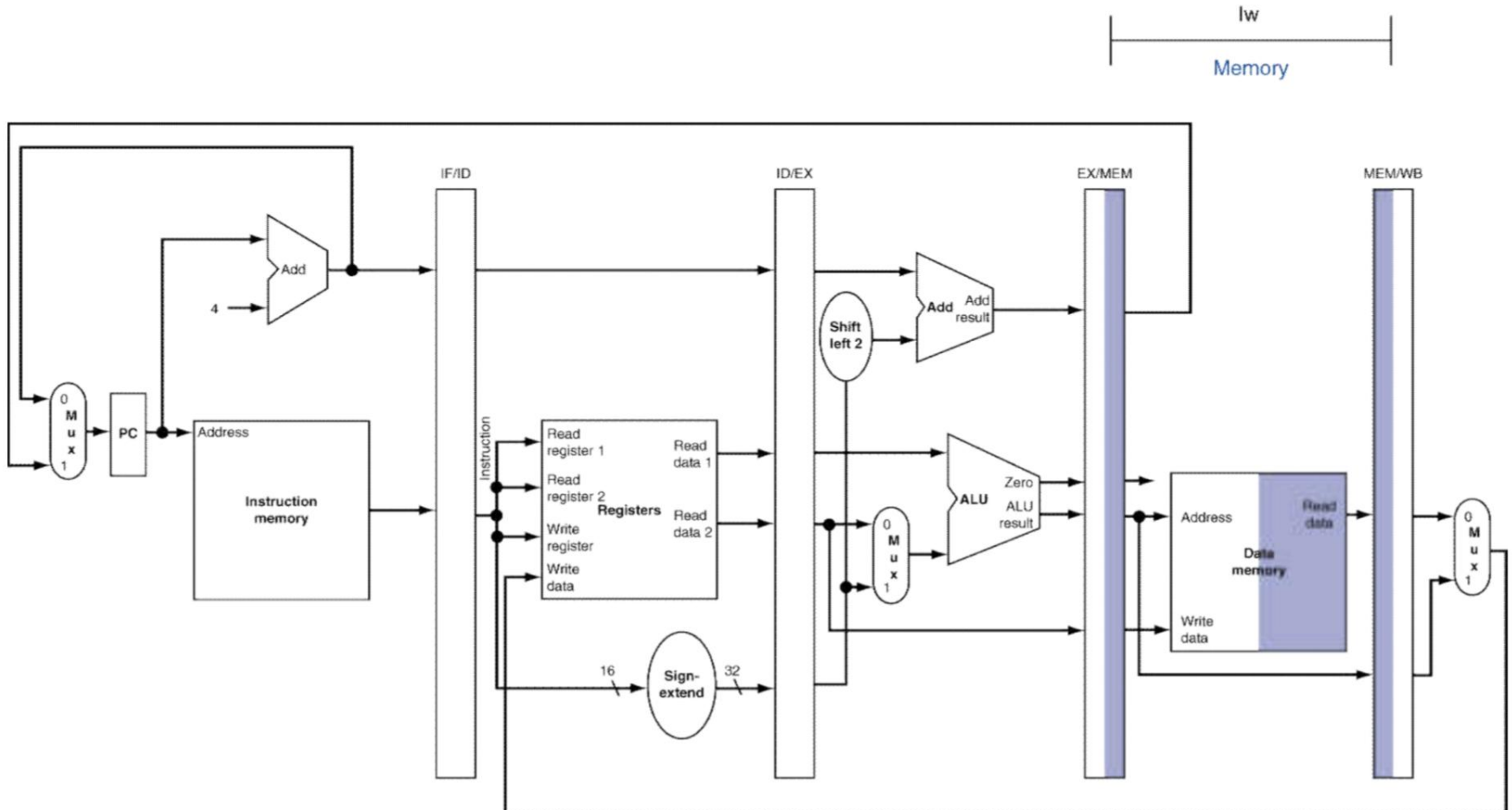
流水线数据通路

- 指令lw的流水线：执行阶段
- EX/MEM: 97位 (PC, ALUOut, Flag, B) sw 4



流水线数据通路

- 指令lw的流水线：访存/分支转移阶段
- MEM/WB: 64位 (Data, ALUOut)



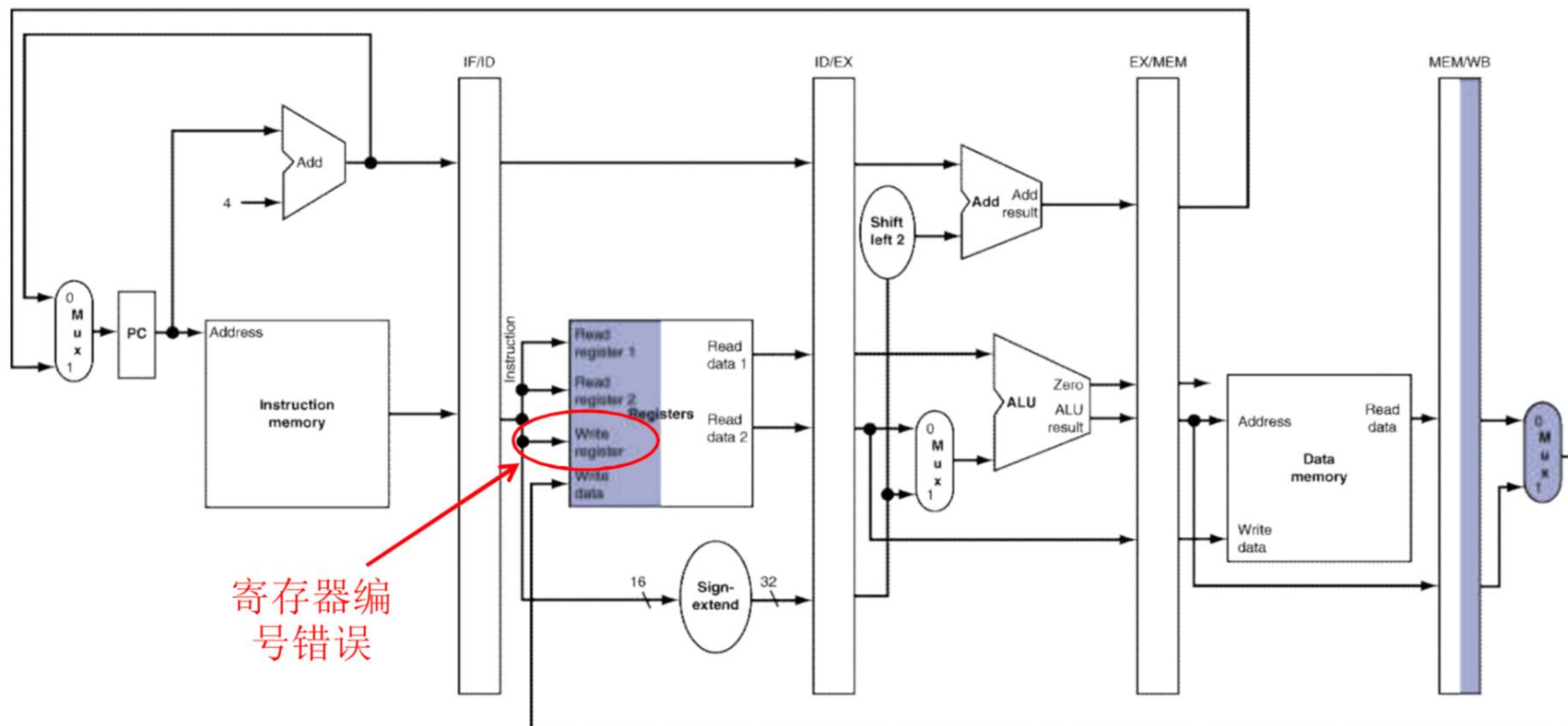
流水线数据通路

- 指令lw的流水线：写回阶段

- MEM/WB: 64位 (Data, ALUOut)

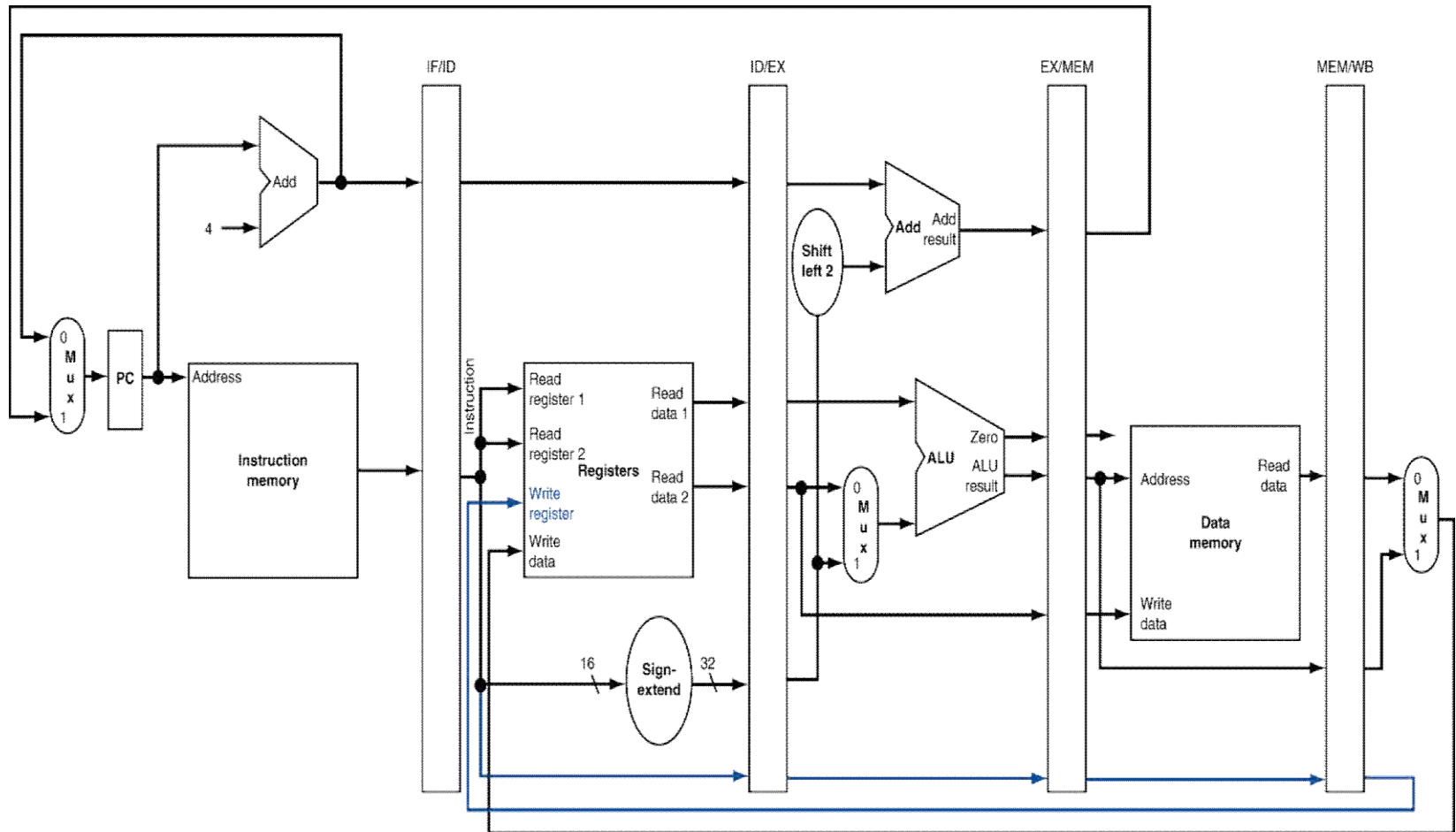
因为IF/ID寄存器已改变了

lw
Write back



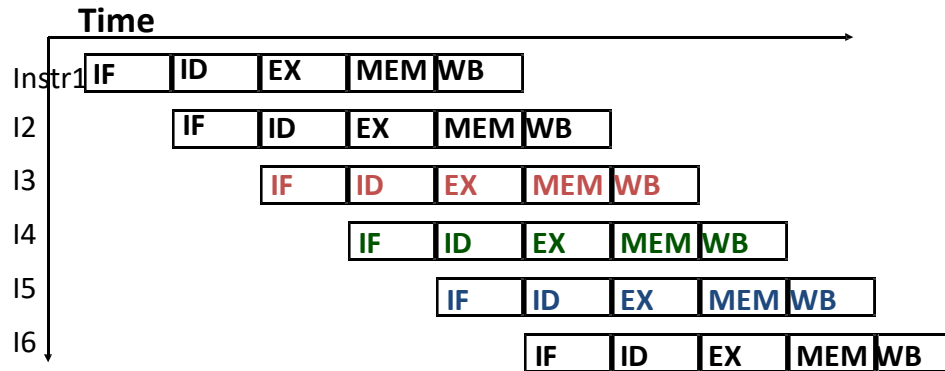
流水线数据通路

- 修正：后三级流水线寄存器增加5位寄存器号



多条指令执行流水线

- 每条指令经过的阶段不同，所以有些指令会在一些阶段空闲出来
 - 例如，R型指令会在Mem阶段空闲
- 规定每条指令都必须经过流水线的每一个步骤，即使什么都没做，否则控制太复杂



时钟驱动的流水线时空图

- 用途：精确分析指令/时间/流水线3者关系
 - 行：某个时钟，指令流分别处于哪些阶段
 - 列：某个部件，在时间方向上的执行了哪些指令
 - 注意区分流水阶段与流水线寄存器的关系 可以看出，在clk5后，流水线全部充满
 - 所有部件都在执行指令：只是不同的指令

相对PC

的地址

偏移

指令

0 Instr 1

4 Instr 2

8 Instr 3

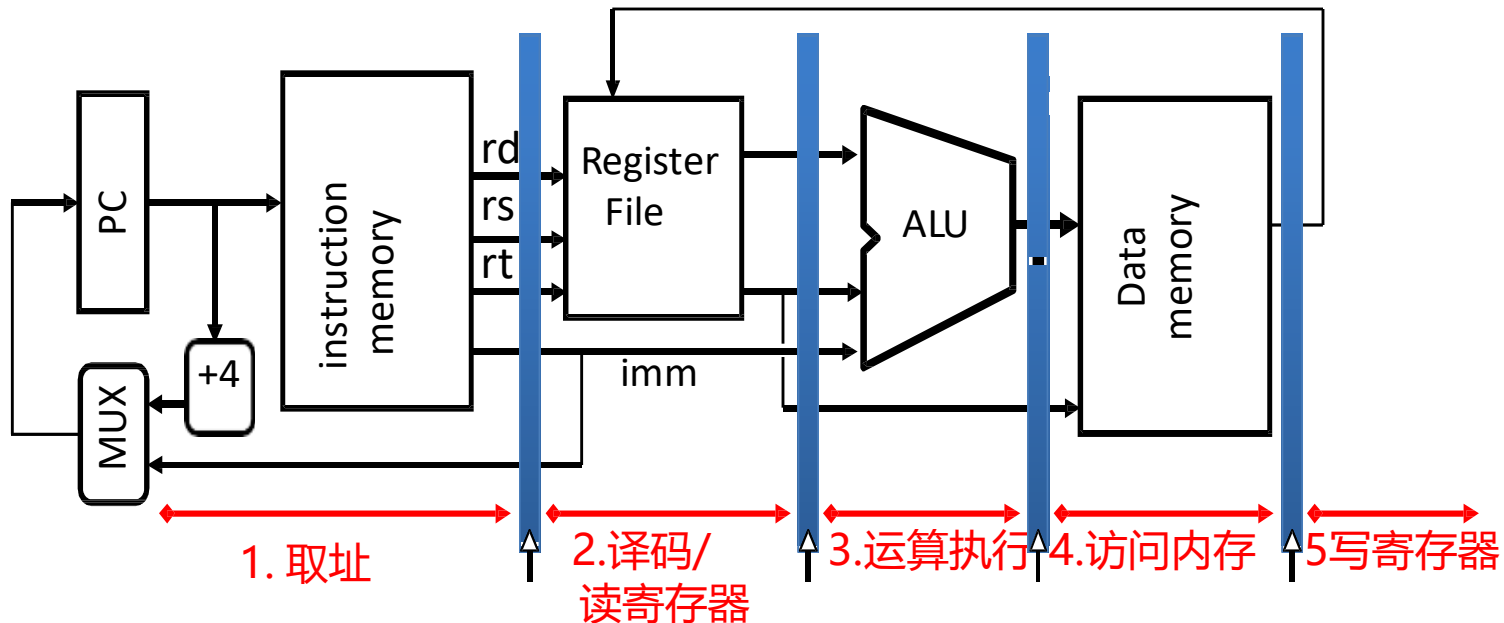
12 Instr 4

16 Instr 5

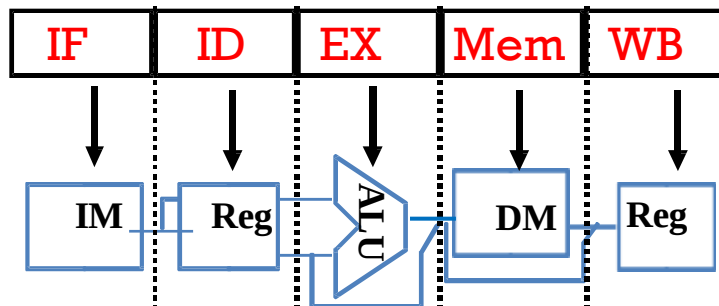
20 Instr 6

			PC	IF/ID	ID/EX	EX/DM	DM/WB	
CLK	PC	指令	取指	译码	执行	访存	回写	
↑ 1	0→4	Instr 1	Instr 1					
↑ 2	4→8	Instr 2	Instr 2	Instr 1				
↑ 3	8→12	Instr 3	Instr 3	Instr 2	Instr 1			
↑ 4	12→16	Instr 4	Instr 4	Instr 3	Instr 2	Instr 1		
↑ 5	16→20	Instr 5	Instr 5	Instr 4	Instr 3	Instr 2	Instr 1	
↑ 6	20→24	Instr 6	Instr 6	Instr 5	Instr 4	Instr 3	Instr 2	

流水线示意图

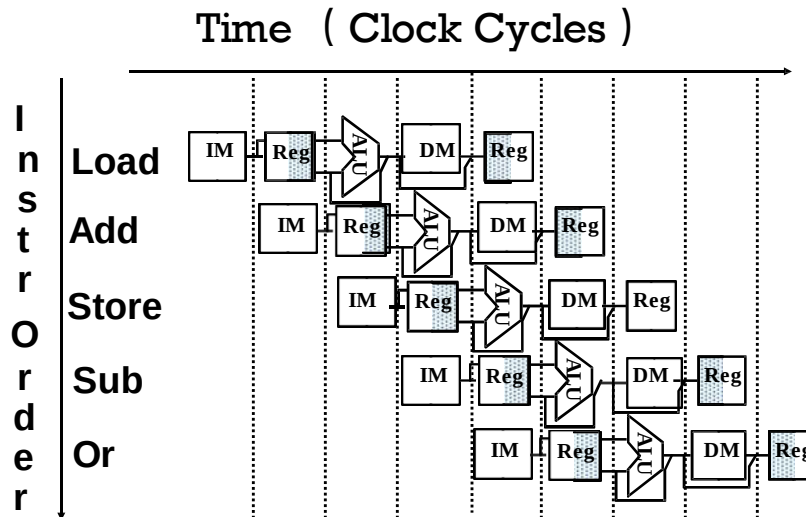


- 使用数据通路关键部件来表示流水线



流水线

- 每个寄存器右部分代表读操作，左部分代表写操作
- 指令集并行： (instruction level parallelism, ILP) 指令流水线技术可以使多条指令在同一硬件上同时执行。

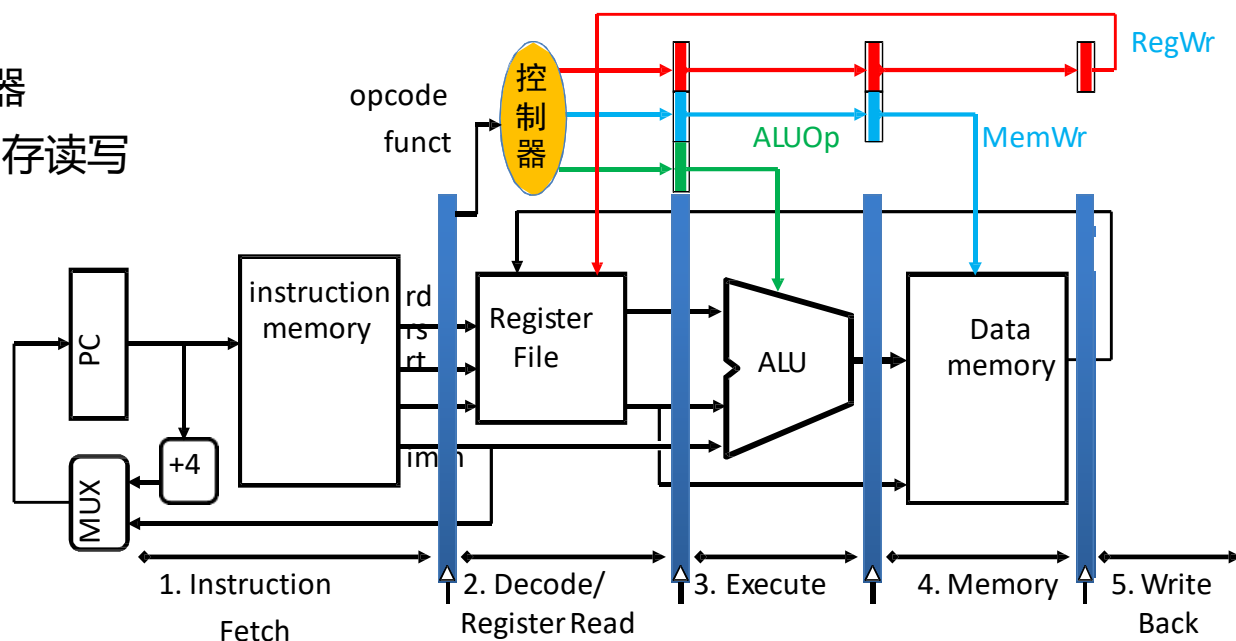


流水线与指令集设计

- MIPS指令集就是专为指令执行的流水线化而设计的
 - 所有的指令均为32位，更易于取址和译码
 - 只有很少的几种指令格式，并且每条指令中的源寄存器字段都固定不变
 - 可以在一个步骤进行译码和读寄存器操作
 - 对内存的操作仅出现在LW/SW操作中
 - 可以在第三阶段进行地址计算，在第四阶段进行存取
 - 内存中的所有操作数必须对齐
 - 不会发生一个数据传输指令访问两次存储器，请求的数据在一级流水内完成传输。

4.6.3 流水线的控制信号设计

- 控制器：译码产生**控制信号**，与单周期完全相同
- 控制信号传递：
 - 扩充流水线寄存器，使之包含控制信号
 - 控制信号在寄存器中传递，直至不再需要
- 控制信号设计：9个控制信号
 - 控制ALU功能
 - 控制多路选择器
 - 控制寄存器/内存读写

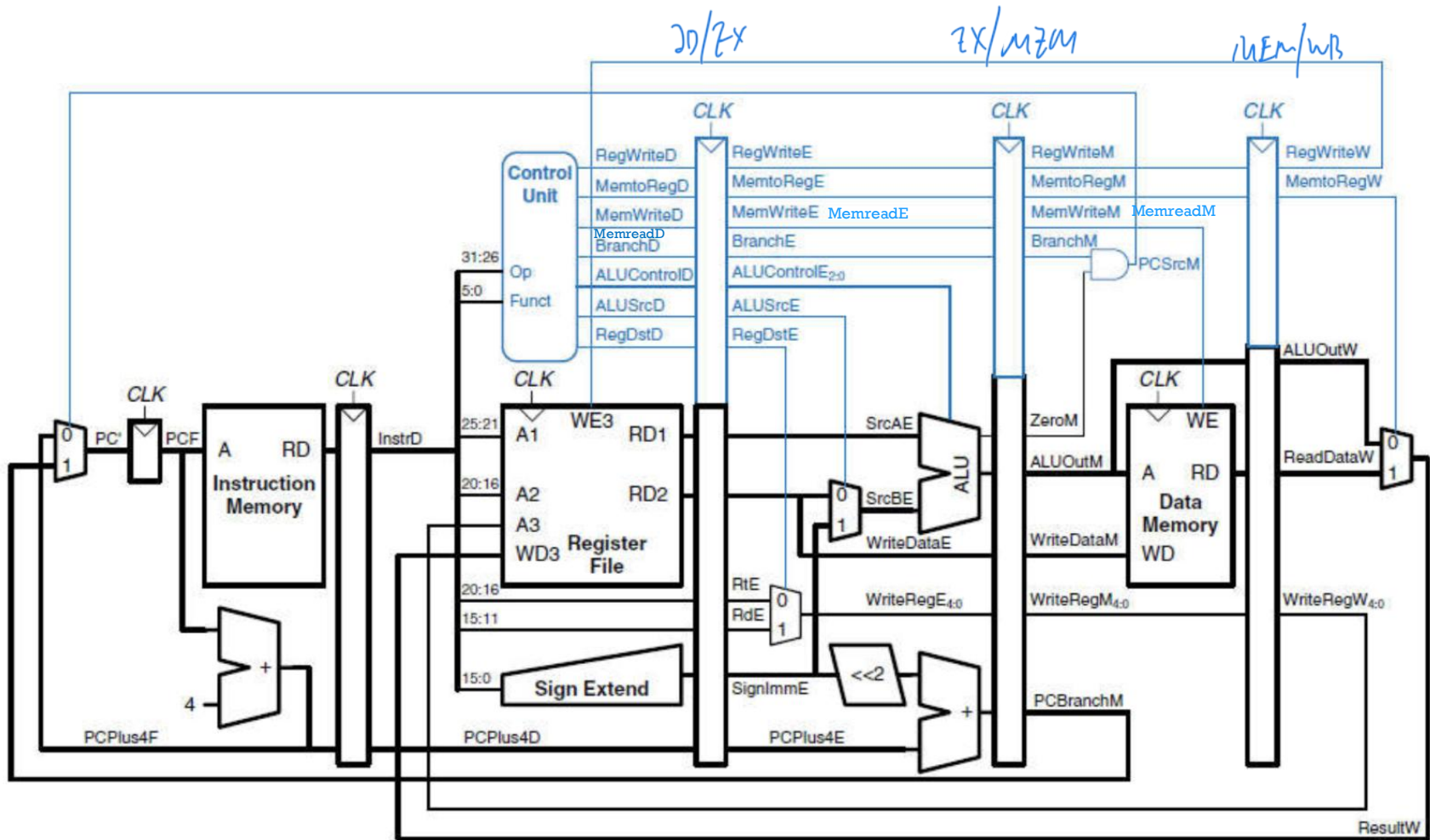


流水控制信号设计

译码后的三个阶段的控制信号

指令	执行计算控制信号				存储器存取控制信号			写回控制信号	
	RegDst	Alucontrol1	Alucontrol0	AluSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R 型	1	1	0	0	0	0	0	1	0
LW	0	0	0	1	0	1	0	1	1
SW	X	0	0	1	0	0	1	0	X
Beq	X	0	1	0	1	0	0	0	X

流水控制信号设计



多周期流水线控制器设计总结

- 硬布线方法
 - 设计、验证CPU各功能模块
 - ALU、寄存器(堆)、主存、辅助计算单元（扩展单元、移位器、加法器等）、时钟
 - 给定指令集，明确每条指令每个周期的数据通路
 - 不用画图的方式（复杂），写到Excel中；
 - 每一对输出-输入，每个多输入处设置一个多路选择器
 - 写出每个寄存器控制端表达式
 - 哪些情况下允许数据进入寄存器
 - 每个Mux输入端聚集，明确几个输入，写出Mux每个选择端子的表达式（可能哪路都不选，输出全0）
 - .v文件：各模块、CU、mipsCPU、initial(给出周期、指令信号)

可用程序完成